

**temperature=0 means the answer is
always identical, right?**



Yes, formally deterministic. But no — not in practice. Why — today's topic.

A simple warm-up question: if you set temperature to zero, will the model's answer to the same query be identical every single time? Intuition says "yes" — zero randomness, always pick the most likely token, what could possibly go wrong. And the intuition is almost right. Almost — because in engineering practice, "almost" costs money: tests that rely on bit-for-bit reproducibility at $T=0$ occasionally fail for no apparent reason, and the cause isn't in your code. Today we'll unpack where this "but no" comes from — along with a few more places where intuition about models holds nine times out of ten, and breaks in the tenth in a predictable, explainable way. We'll walk the inference pipeline from text to answer, and at every stage look at where intuition built from observation stops being a reliable map.

LECTURE 2

02

How Modern Large Language Models Work

The inference pipeline — and the boundaries that change engineering decisions



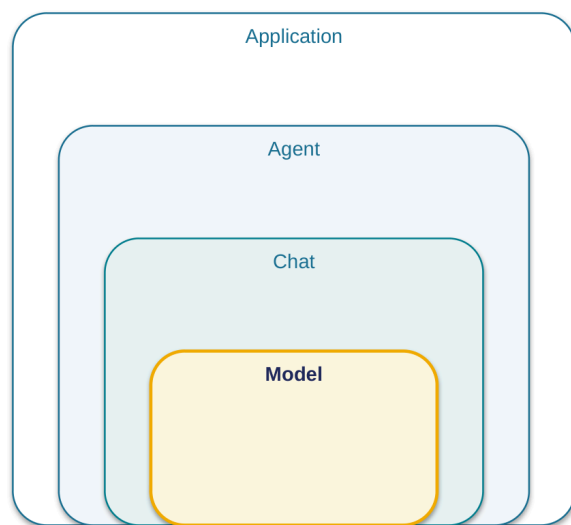
This is the second lecture of the course. Today we'll look at how a large language model works on the inside: we'll systematize the pipeline's structure and unpack the nuances that directly affect engineering decisions. If you work with models daily, this will refresh the fundamentals and add nuance; if these concepts are new to you, you'll get everything you need. The lecture's backbone is the inference pipeline: the path of a single request from text to answer. Text gets cut into tokens, tokens turn into vectors, the attention mechanism decides which parts of the context matter, a token gets picked from a probability distribution — and the cycle repeats. Four stages, four sections; a fifth section — new in this version of the course — covers model types and sizes: what they run on and what they can do; a sixth section assembles the pipeline as a whole and wraps up. Let's move on to the map.

Lecture map — 7 sections

- 0 Introduction** — *the frame and the pipeline as a whole*
- 1 Tokenization** — *how the model sees your text*
- 2 Embeddings** — *the space of meaning and the boundary of similarity*
- 3 Attention Mechanism** — *what matters right now: roles, caching, long context*
- 4 Sampling** — *from distribution to token: temperature, determinism, invisible tokens*
- 5 Model Types and Sizes** — *what models run on, multimodality, the 2026 landscape*
- 6 Wrap-up** — *assembling the pipeline, recap of the mechanisms*

A quick route map before we dive in. The lecture follows the inference pipeline: the order of sections mirrors the order of stages every one of your requests passes through. Section 1 — tokenization: how the model sees text, why a "fixed" strawberry answer proves nothing, where number-chunking and glitch tokens come from. Section 2 — embeddings: the space of meaning, three different things that all get called "embedding," and the boundary where semantic similarity stops meaning usefulness. Section 3 — the attention mechanism: why a role-based prompt genuinely changes the answer; where caching economics comes from; what a one-million-token window can actually do. Section 4 — sampling: temperature, the reason $T=0$ doesn't give you determinism, structured output, and the invisible reasoning tokens you're paying for. Section 5 — new in this version of the course: model types and sizes — what a small model runs on versus a giant, what each class can do in terms of multimodality, and the landscape of specific models as of September 2026. Section 6 — wrap-up: the pipeline as a whole and a recap of the mechanisms we covered. Keep the map in mind — it will keep you from getting lost in the details.

Today's object — the “model” layer from Lecture 1's four layers



The “model” layer: stateless inference — input goes in, a prediction comes out, no memory between calls.

Today: we take apart what happens inside that inference — and where its design changes engineering decisions.

Today's object of conversation is the "model" layer. Lecture 1 gave us a layered picture: at the bottom, the model; above it, the chat loop with a system prompt and history; above chat, the agent loop; on top, the application. There, the model was described as stateless inference: input goes in, a prediction comes out, no memory between calls. From that same lecture we already have the context window as a constraint, the distinction between local and cloud models, and Pearl's three levels of causation — all of that is taken as known today and used without re-deriving it. With that description we can already talk about where the model sits in the larger system. But the question of what exactly happens inside that function between request and answer — Lecture 1 deliberately left that a black box. Today we open that box, and everything that happens in this lecture happens inside a single layer, the bottom one. Chat, agents, RAG, and applications are built on top of it and are discussed in Lecture 3 and beyond. We're chasing boundaries: the places where the model's internal design changes how you build prompts, agents, and decisions. Every mechanism in this lecture was chosen because it has an observable engineering consequence — in cost, speed, quality, or reproducibility.

Lecture goal

“Examine how a language model works — and get into the **important details that change how you build prompts, agents, and decisions.”**

why a fixed “strawberry” answer proves nothing

why a role-based prompt genuinely changes the answer

what a 1M-token window can actually do

why $T=0$ doesn't guarantee identical answers

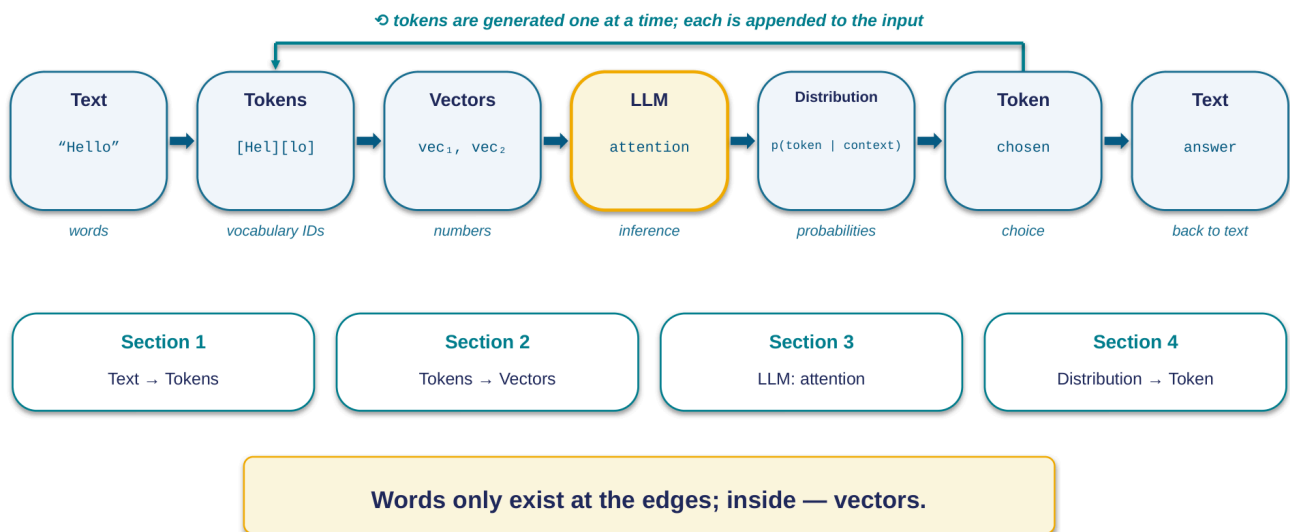
what invisible reasoning tokens actually cost

small model vs. giant — what criterion to choose by

what to replace blind trust in benchmarks with

The goal of this lecture: examine how a language model works — and get into the important details that change how you build prompts, agents, and decisions. We'll walk the inference pipeline from text to answer, and at every stage — alongside the mechanism — we'll show its boundaries. We'll show exactly where tokenization breaks arithmetic and why a “fixed” strawberry answer proves nothing. We'll show the mechanism that makes a role in the prompt genuinely change the answer. We'll unpack why temperature=0 doesn't give you reproducibility and what it actually costs to get real reproducibility. On top of that, we'll add the real cost of invisible reasoning tokens, the criterion for choosing between a small model and a giant, and what to replace blind trust in benchmarks with when picking a model. Seven lines under the goal — one per section we'll cover today: each one will get an answer through a concrete mechanism by the end of the lecture, not through intuition. If at any point it looks like some mechanism doesn't affect practice, call me out on it — we don't cover those today.

Data flow in an LLM — there and back again



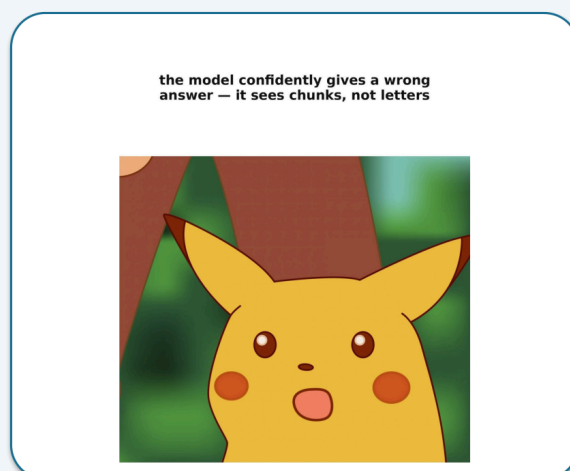
This is the keystone diagram of the whole lecture — the inference pipeline, the path of a single request from text to answer. Let's look at it as a whole before the first deep dive; we'll come back to it at the start of every section. On the left is your text. The first transformation is tokenization: text is cut into tokens, IDs from the model's vocabulary; that's Section 1. Next, each token turns into a vector — a list of numbers the neural network can compute with; that's Section 2. In the center is the model itself: the attention mechanism decides which parts of the context matter for the next step; that's Section 3. On output, the model doesn't produce an answer — it produces a probability distribution over the entire vocabulary; one next token is picked from it, gets appended to the context, and the cycle repeats; that's Section 4. The last step: the chosen tokens are assembled back into text. The key observation: words only exist at the edges of the pipeline — at the input and the output. Everything inside is operations on vectors. And notice the loop in the diagram: tokens are generated one at a time, each chosen token gets appended to the input, and the pipeline runs again; the answer is built through successive turns of that loop. This axis isn't just for today. Every later lecture in the course sits on the same axis: RAG is about managing what makes it into the context before the pipeline; agents are a loop wrapped around the pipeline; cost optimization is the pipeline's economics. Today's lecture moves along this same axis — each section unpacks its own stretch of the pipeline along with the boundaries of its applicability.

Section 1

Tokenization

"How the model sees your text"

3 case studies · 3 failures



We're entering the pipeline's first stage — tokenization. This is the layer closest to you: it's the first thing your text hits the moment you hit "send." Text doesn't enter the model as letters or words — it's cut into tokens, statistically frequent subsequences from the model's vocabulary. You already know the definition of a token, so we'll move through the basics quickly and precisely — we need it as common ground. Most of the section's time will go to the second floor: the places where frequency-based cutting diverges from the structure of your data. We'll look at the 2026 diagnosis of the classic letter-counting meme: what exactly does a model prove when it gets strawberry right and gets cranberry wrong. We'll see what the tokenizer does to numbers and code and why that breaks arithmetic. We'll meet glitch tokens — undertrained corners of the vocabulary that haven't gone away even in current models. And we'll close the section with economics: how much the same text costs in different languages, and what to do about it when setting limits. The section's general principle: every tokenization oddity has a mechanism, and every mechanism has a verifiable engineering consequence.

A token is an ID from the model's vocabulary — not a letter, not a word

`cat` → `[cat]` → 1 token

`hyperparameter` → `[hyper][param][eter]` → 3 tokens

`tokenization` → `[token][ization]` → 2 tokens (o200k_base)

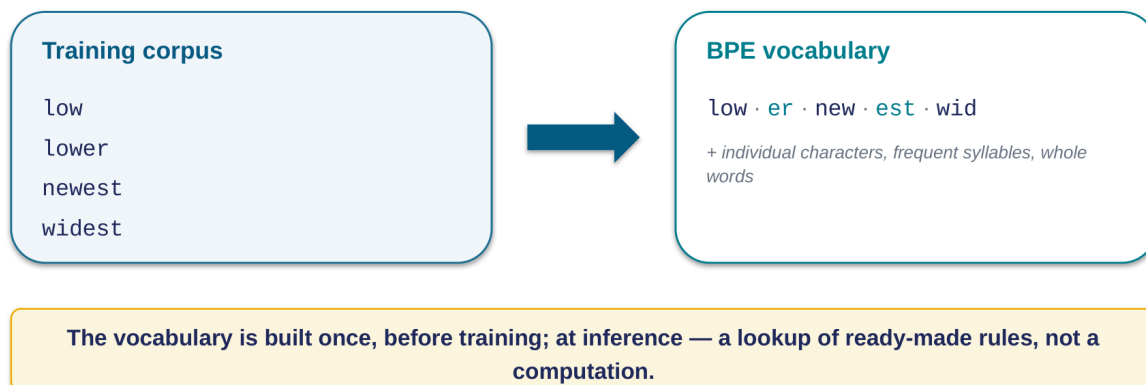
“On average: 1 token ≈ 4 characters in English (≈ 2 characters in Russian)”

Vocabulary and model are two separate artifacts: the vocabulary is built before the model is trained, by a separate algorithm on its own corpus.

The precise definition. A token is an identifier, an integer from the model's vocabulary; the vocabulary is fixed at training time and doesn't change at inference. A token is not a letter and not a word — it's a statistically frequent subsequence of characters, learned from a corpus. The common English word `cat` makes it into the vocabulary whole; `hyperparameter` splits into three tokens; `tokenization` splits into two, in that same o200k_base vocabulary. Vocabulary sizes for current models run in the hundreds of thousands of entries: around 200,000 for the GPT-4o family and newer, around 100,000 for earlier generations, 128,256 for Llama 3 and newer. A working cost estimate: one token is roughly four characters of English text and roughly two characters of Russian text. One clarification that often gets lost: the vocabulary and the model are two separate artifacts. The vocabulary is built by a separate algorithm on its own corpus before the model is trained; the model then learns to work with that vocabulary on its own corpus. This decoupling is the source of a whole class of effects we'll come back to in this section. And the answer to the reasonable question "why not just work character by character": length economics. A character-level representation lengthens the input three to five times over, and attention cost grows quadratically with length — a four-times-longer input means a sixteen-times-more-expensive attention layer. Tokenization, with all its artifacts, is a deliberately chosen trade-off, not an oversight that will get "fixed in the next version."

BPE — a compromise between an alphabet and a full-word vocabulary

Not all letters (too long) and not all words (unfamiliar ones fall through) — frequent subsequences.



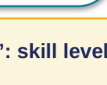
Different vendors cut the same text differently: Claude, GPT, Gemini all have their own vocabularies and rules.

The algorithm behind most current LLM vocabularies is BPE, byte-pair encoding. It's a compromise between two extremes. A vocabulary of individual characters can represent any text, but the resulting sequences are long. A vocabulary of whole words gives short sequences, but any unfamiliar word — a typo, a neologism, a name — falls through. BPE sits in the middle: start with the alphabet, iteratively merge the most frequent adjacent pairs; after many iterations the vocabulary contains individual characters, frequent syllables, and whole high-frequency words all at once. The key engineering detail: the vocabulary is built once, before the model is trained. BPE is run on a corpus, the vocabulary and merge rules are fixed — and that's it; at inference, tokenization is a lookup against a ready-made table, taking milliseconds. That gives us two consequences we'll use later: the tokenizer cuts text based on the frequency statistics of its own corpus, and those statistics may not match the structure of your task; and since the vocabulary and the model are trained on different corpora, the vocabulary can end up with entries the model has barely ever seen. Another practically important point: different vendors count the same text differently — their own vocabularies, merge tables, whitespace handling. The same document will produce a different token count with different providers, meaning different cost and different context-window usage. When moving workloads between providers, recompute your token budget: for OpenAI's models, tokenization is emulated locally with the tiktoken library; for open models, with the tokenizers library; Claude's tokenizer is closed, but actual usage is visible in the counters on any API response.

The patch race: they fixed strawberry, then cranberry — one word at a time

Mechanism — letter blindness

strawberry → [st][raw][berry] The model sees **3 tokens**, not 10 letters.

GPT-5.2: 2 r's in strawberry	
GPT-5.5: strawberry ✓ / cranberry ✗	
GPT-5.6: cranberry ✓	
StrawberryBench: still 60%	

GPT-5.2 · Dec 2025

strawberry ✗ — "two r's in strawberry"

GPT-5.5 · Apr 2026

strawberry ✓ / cranberry ✗ — "two r's" instead of three

GPT-5.6 · Jul 2026

cranberry ✓ — another viral case patched

StrawberryBench

847 questions, 7 difficulty levels — systematic testing instead of one viral question

"Jagged intelligence": skill level is set by training data, not by "general intelligence" — the same model wins Olympiad gold and fails letter counting.

What to do: test your own domain's "cranberry" — obscure cases nobody hyped about; going viral ≠ having the skill.

You know the "how many r's in strawberry" meme; you probably also know that current models answer it correctly now. The conclusion "so they've learned to count letters" is wrong, and here's why. The mechanism is letter blindness. A word arrives at the model as three tokens, not ten letters. Inside the learned token vector there's no field saying "contains an r at position N" — there's contextual statistics instead.

Character-by-character counting requires reasoning on top of the tokenized representation — spelling the word out or calling code; a single forward pass doesn't reliably get you there. This is a structural property of the pipeline. Now look at the 2026 patch race. GPT-5.2, in December 2025, still answered "two r's" for strawberry. GPT-5.5, released in April 2026, passes strawberry — but on "how many r's in cranberry" it answered "two"; the correct answer is three. Only GPT-5.6, in July 2026, finally fixed cranberry too — check the current model right now. Notice the pattern: each viral case gets patched individually, one at a time, rather than through a single skill-level improvement — otherwise fixing strawberry would have immediately closed cranberry too. To test the task class systematically, StrawberryBench appeared — 847 questions across seven difficulty levels; it's there, not on one viral word, that you see character counting remains a genuine weak spot. The phenomenon has a name: "jagged intelligence" — a capability profile with sharp dips right next to peaks. The reason is that skill level is set by training data, not by "general intelligence": wherever a skill is richly represented in the data, there's a peak; wherever the word's representation hides the needed structure, there's a dip. The same model solves Olympiad-level math and fails letter counting; on multiplication, GPT-4 without tools scored around 59% on three-digit numbers, 4% on four-digit numbers, and 0% on five-digit numbers. The lesson carries over to practice: if a model passes your test, check it against "your domain's cranberry" — a structurally analogous but unhyped example nobody tested on: going viral is not a skill. And for character-level operations and arithmetic — use an

The tokenizer cuts by frequency, not by structure

Numbers and code are the most common "non-text" inputs: the model's arithmetic and your token budget both depend on how they're cut.

Numbers

1000000 → [100][000][0]

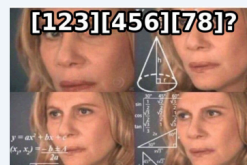
Chunks left-to-right ≠ place values, which are read right-to-left: boundaries don't line up, and column addition breaks.

Right-to-left cutting improves arithmetic; task-specific schemes give **up to +33% accuracy** over standard cutting.

Code

GPT-2: 16 tokens for a level-4 indentation.

GPT-4: groups whitespace — the vocabulary gets fixed, but not for every task at once.



What to do:

Digit separators ("1,234,567")

Offload computation to a tool

Consistent indentation

Off-the-shelf chat products already route counting to built-in tools (code interpreter) automatically; call a tool yourself for non-standard cases and your own apps built on the API.

Why should an engineer care how numbers and code get cut: these are the most common "non-text" inputs, and how they're cut directly affects the model's arithmetic and your token budget for code. Two compact examples of how frequency-based cutting diverges from data structure. Numbers. The cl100k_base tokenizer standardized cutting numbers into chunks of three digits left-to-right: a million turns into groups whose boundaries don't line up with place values. Humans read place values right-to-left — thousands, millions; the model gets irregular blocks — and that's a direct source of some arithmetic errors. Research confirms the diagnosis from both sides: forcing right-to-left cutting noticeably improves numerical reasoning, and task-specific number-tokenization schemes gave up to thirty-three percent more accuracy on large-number arithmetic relative to standard cutting. Code. GPT-2 encoded every indentation space as a separate token: a line at the fourth nesting level spent sixteen tokens just on indentation. GPT-4-generation tokenizers group whitespace into single tokens, specifically targeting Python style. This is a rare case where a tokenization problem got fixed by changing the vocabulary — a useful contrast to strawberry: you can optimize the vocabulary for a frequent input class, but you can't align cutting with the structure of every task at once. Four takeaways. Write significant numbers with digit separators — "1,234,567" instead of an unbroken string: separators align token boundaries with place values. Offload any arithmetic beyond a rough estimate to a tool — models write excellent code for calculations they can't do themselves. For code — use consistent formatting, and estimate your budget by measuring on real files: the common "four characters per token" rule of thumb is systematically wrong for code. And an important caveat for practice: in off-the-shelf chat products — ChatGPT, Claude's web interface — counting and arithmetic already route automatically to a built-in tool like a code interpreter, bypassing the model's direct forward pass; what we just covered becomes critical specifically when you call the model directly through the

API — in your own apps and agents, where there's no automatic routing like that, and in non-standard cases that the off-the-shelf product didn't recognize as "needs a tool."

Roughly 4% of the vocabulary are glitch tokens

SolidGoldMagikarp (2023)

A Reddit username that ended up in GPT's vocabulary — the model couldn't repeat it (the token's name is a reference to the Pokemon Magikarp).



Mechanism: vocabulary corpus ≠ model training corpus
→ the token's embedding stays at its random initialization.

Practical impact

- Parsing failures on exotic strings — rare identifiers, obfuscated text, unusual Unicode
- Production risks — logs, user-generated IDs, arbitrary user input
- Unexplained off-topic answers with no code error

GlitchMiner (AAAI 2026)

roughly 4% of the vocabulary by one estimate;

reproduced in the open Llama, Qwen, Gemma, Phi-3, Mistral families.

In practice:

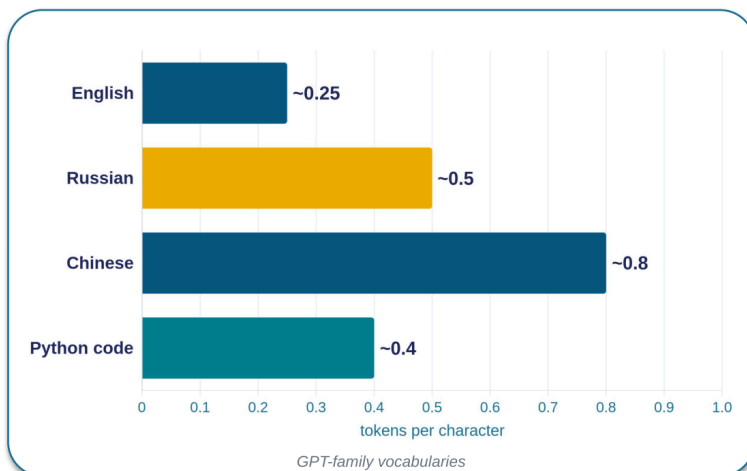
Unexplained behavior on exotic strings (rare identifiers, obfuscated text, unusual Unicode)? Hypothesis: a glitch token. Test: replace the string with a placeholder.

In products accepting arbitrary input — normalize and sanitize input before feeding the model.

A systemic property of the pipeline, not a version bug — it doesn't get fixed by scale.

In January 2023, researchers studying clusters in the embedding space of GPT models stumbled onto a group of odd vocabulary entries — strings like SolidGoldMagikarp: Reddit usernames that had ended up in the tokenizer's corpus during data collection. Models behaved anomalously on these tokens: they couldn't repeat them, answered off-topic, drifted away from the conversation. The mechanism, briefly: a string can be frequent in the corpus the vocabulary was built from, and earn its own token — while barely appearing in the corpus the model was later trained on; that token's embedding stays close to its random initialization and "means nothing" in the learned geometry. 2026 data says: this isn't a quirk of the GPT-3 era, it's a systemic property. By one estimate, roughly four percent of the vocabulary entries in tested models are glitch tokens, and the GlitchMiner framework finds them via gradient search across ten open LLM families — Llama, Qwen, Gemma, Phi-3, and Mistral. The problem reproduces across every family tested, because the vocabulary is built separately from the model; scale doesn't fix it. Now — what this actually affects in engineering practice, and that matters more than the mechanism itself. First: parsing failures on exotic strings — the model unexplainably loses the thread of a conversation or fails specifically on one input, not across a whole task class. Second: production risk — any system accepting arbitrary user input works with a potential source of glitch tokens: logs, auto-generated IDs, obfuscated text, rare Unicode sequences. Third: diagnosis — if behavior is unexplainable on one specific input, "there's a glitch token in the input" should be among your hypotheses; it's testable in a minute by swapping the suspicious string for a placeholder — behavior corrects, culprit found. Fourth, product-level: normalize and sanitize input before feeding the model — this catches not just glitch tokens but a whole class of related tokenization surprises.

Russian text costs roughly 2× more than English



Move to o200k_base:

roughly **−35%** for non-Latin languages — the gap narrows but doesn't disappear.



What to do:

- Calibrate any token-based limit for your own language: retrieval chunks, max_tokens, context-window budget.
- For batch processing of large volumes — consider translating to English (≈2× cheaper); in interactive work the difference isn't worth it.

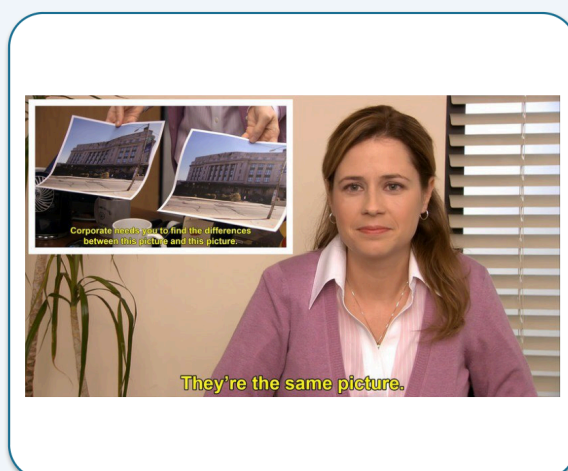
The same text, in terms of meaning, costs a different number of tokens depending on the language — a direct consequence of the fact that a BPE vocabulary is learned from a corpus dominated by English. Rough benchmarks for GPT-family tokenizers: English is around 0.25 tokens per character, Russian around 0.5, Chinese around 0.8, Python code around 0.4. So a Russian-language request costs roughly twice as much as an English one, with the ratio ranging from about 1.5x to 2.5x, and it burns through the context window twice as fast: an 80,000-character document is roughly 20,000 tokens in English and 40,000 in Russian. The 2024-2026 trend: OpenAI's move to o200k_base cut the per-unit cost of non-Latin languages by roughly 35% — the gap narrows but doesn't disappear: English remains the statistical core of the corpora. For models with a larger share of Russian in their vocabulary — YandexGPT, GigaChat — the gap is smaller; that's one rational argument in their favor for high-volume Russian-language workloads. The rule stays the same: for batch processing of large volumes, evaluate whether the task tolerates translating to English; in interactive work, the difference usually isn't worth the loss of convenience. The language coefficient also leaks into places people forget about. Splitting documents into retrieval chunks is configured in tokens — a "512-token" chunk in Russian holds half the meaning, and thresholds from English-language guides are systematically too small for a Russian knowledge base. The max_tokens limit is the same story: a Russian answer runs longer in tokens, and mid-sentence truncation happens more often. Calibrate any numeric, token-based parameter against your own language and your own data.

Section 2

Embeddings

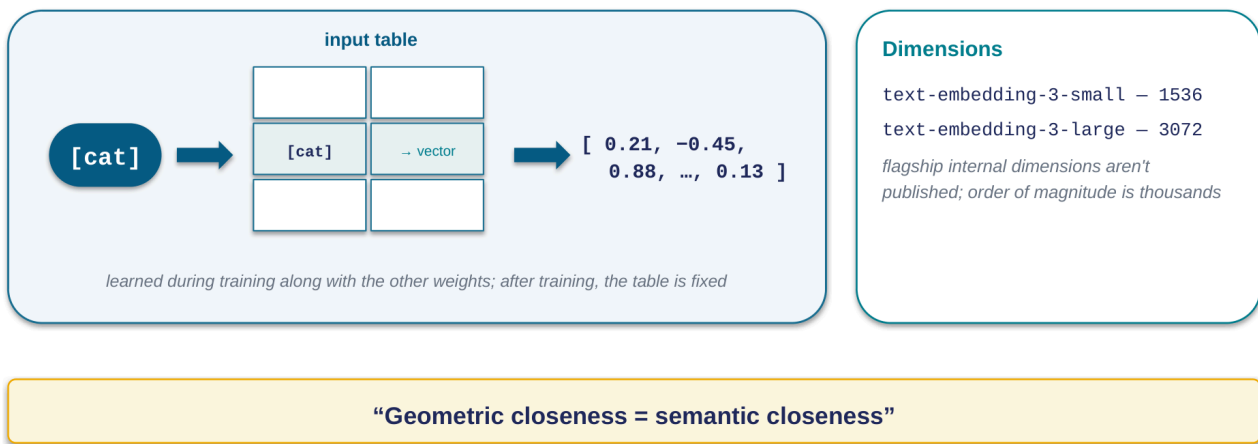
"The space of meaning — and where similarity breaks down"

4 case studies · 1 failure



The first stage is behind us: text became a sequence of vocabulary identifiers. But a neural network can't meaningfully work with a bare number — there's no meaningful arithmetic between token IDs. The second stage of the pipeline turns identifiers into a representation you can actually compute with — vectors. Here's the plan for this section. First, we'll pin down what an embedding actually is: a vector lookup from the model's input table — quick, this is the basics. Then — the section's main organizing move: the word "embedding" in engineering usage refers to three different things, and they're often conflated; we'll carefully unpack the three lives of the term — and see that embeddings work not only inside inference but also as a standalone search tool. After that — the space of meaning: how coordinates get learned, why texts close in meaning end up near each other, and how that's measured via cosine similarity. Then — the section's one important failure case: the similarity boundary, where high similarity delivers a document with a nearly opposite practical meaning, and what production search systems do about it. And we'll close the section by tying it together: why embeddings are the foundation of the model "understanding" rephrasing, and what that implies for search systems and RAG, which we'll get to in Lecture 3.

Every token gets a vector from the model's input table



What to do: a typo or a different letter case is already a different token and a different vector; normalize input before embedding it.

A token is a vocabulary identifier, but a neural network can't meaningfully work with the number 48,213: there's no meaningful arithmetic between token IDs. An embedding — the vector representation — is a fixed-length vector assigned to every vocabulary token, a list of floating-point numbers, learned during training along with the rest of the weights. After training, the "token → vector" input table is fixed; at inference time, the model does a lookup: it gets an identifier, pulls the vector, passes it along. The key property of the learned space: geometric closeness corresponds to semantic closeness. "Cat" is close to "dog," "SSL" is close to "HTTPS" — not because someone labeled it that way, but because those words appeared in similar contexts in the training corpus. Closeness in this space is a statistical reflection of usage, not a semantic reference. Dimensionality is a hyperparameter: OpenAI's public embedding models use 1536 and 3072 dimensions; flagship LLMs' internal dimensions aren't published, but the order of magnitude is thousands. And one caveat for anyone who'll work with vectors directly. Picture a three-dimensional space where words with similar usage sit close together, and stretch the number of dimensions to the thousands. Most of your closeness intuitions carry over, but in high-dimensional spaces concentration of measure kicks in: random points end up at roughly the same distance from each other, and "raw" distances compress into a narrow range. So absolute similarity values aren't very informative on their own — what's informative is comparisons and the distribution of values within your specific task. This is the first reason universal similarity thresholds don't exist; the second, more substantive reason, is coming up.

Embeddings aren't just an inference internal — they're also a search tool

When you build search or RAG, you're using the third life of the term "embedding" — a separate embedding model.

1

The input lookup table

inside inference

Static: the vector for [cat] is the same in every sentence. A lookup by ID, before any context exists.

2

The model's internal data representation

inside inference

Vectors the model recomputes as it reads context — after the attention layers. These are what carry the model's "understanding."

3

Vectors for search

a standalone tool

A vector for a whole text, from a separate embedding model — not the internals of your chat LLM. Its own product, its own training, its own leaderboards.

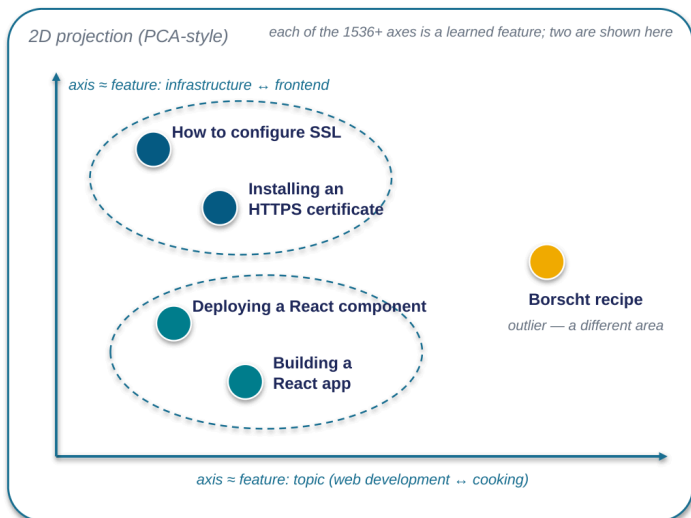
this is your search/RAG

Updated the chat LLM → you do NOT need to re-index the database: the index lives in the embedding model's coordinate space.

Embeddings aren't just for inference — they're also a standalone search tool. In engineering usage, the word "embedding" refers to three different things; let's unpack the three lives of the term and see which one you're actually using when you build search. Life one: the input lookup table — a static "token → vector" table at the LLM's input. The vector for the token "cat" is the same one, regardless of sentence. Life two: the model's internal data representation. The sequence of input vectors passes through dozens of attention layers, and at each layer, every position's vector is updated to reflect its surroundings — by the output, the vector at the position "cat" no longer equals the table entry, context has been mixed in; these are the vectors the model recomputes as it reads context. It's these representations that carry the model's "understanding" — the input table is just a starting point. Both of these lives are internal to inference. The third life is your working tool: when you call an embedding API for search or RAG, you get a vector for an entire text from a separate model, specifically trained for search and comparison. This isn't the internals of your chat LLM: the embedding model is a standalone product, and the chat model and the embedding model can come from different vendors entirely — that's normal practice. The rule for keeping these apart: if it's about cost and context window — that's tokens and the input table; if it's about "what the model understood" — that's the internal data representation; if it's about search and vector databases — that's the output embeddings of a separate model. And the typical mix-up mistakes this distinction exists to prevent. "We already pay for a chat-model subscription — why pay for an embedding model too?" — because the chat model doesn't give you search vectors. "Let's use an embedding model from the same vendor — they'll be more compatible" — there's no such thing as compatibility here as a category: vectors are only comparable against other vectors from the same embedding model; the LLM never even sees them. And, mirroring that: "we updated the LLM — we need to re-index the database" — no you

don't; re-indexing is needed exactly when the embedding model itself changes.

Tokens close in meaning sit near each other — across hundreds to thousands of dimensions



Dimensionality

Public embedding models: **1536–3072** dimensions; flagship internal dimensions are on the order of thousands.

Training

Coordinates aren't hand-assigned: similar usage contexts → close vectors.

Projection

You can only view the space through PCA/t-SNE — the 2D picture loses some of the structure.

What to do: closeness in this space is measurable as distance — filtering and clustering without labels and without an LLM is possible directly on the vectors, cheaply.

How the space where these vectors live is structured. Every point is a token or a text; the coordinates are those same hundreds or thousands of numbers. The meaning of the dimensions isn't hand-assigned — they're directions the model learned from statistics; after the fact, many of them can be read as features: topic, formality, domain. The training objective arranges the space so that units that appeared in similar contexts end up close together. "How to configure SSL" and "Installing an HTTPS certificate" are surrounded by the same words — "certificate," "server," "domain" — and their vectors come out close; "Deploying a React component" and "Building a React app" sit next to words about frontend and builds — and they too end up near each other; while "Borscht recipe" lives in a completely different region of the space — a lone outlier among technical texts. The structure emerges not from rules but from millions of usage examples. On the left is a two-dimensional projection: two semantic clusters and an outlier — the same five texts you'll see again in the pairwise similarity table coming up. The projection's axes are labeled as features: one separates web topics from cooking, the other separates infrastructure from frontend; each of the real space's 1500-plus axes is a learned feature just like these, and here we're showing two of them. Remember this is a simplification: the real projection comes from a space with thousands of dimensions, via algorithms like principal component analysis or t-SNE, which pick the two axes with the most variance. Any flat picture loses part of the multidimensional structure — it's good for intuition, not for conclusions. And let's repeat the caveat, because it costs money in practice: in high-dimensional spaces, "raw" distances compress into a narrow range, and an absolute similarity value on its own says little. What works is comparisons — "this document is closer than that one" — and the distribution of values within your specific task. A universal threshold like "above 0.8 means similar" doesn't exist; the threshold is calibrated empirically on your own data. Next, let's see how this geometry works on whole sentences — and where its boundary

lies.

High similarity means “about the same thing,” not “with the same meaning”

cosine similarity	SSL	HTTPS	React c.	React a.	Borscht
How to configure SSL	1.00	0.85	0.18	0.20	0.08
Installing an HTTPS certificate	0.85	1.00	0.22	0.19	0.07
Deploying a React component	0.18	0.22	1.00	0.78	0.12
Building a React app	0.20	0.19	0.78	1.00	0.10
Borscht recipe	0.08	0.07	0.12	0.10	1.00

“How to configure SSL” ↔

“How to disable SSL”

Very high similarity — opposite practical meaning.

scale: 0 — light · 1 — dark

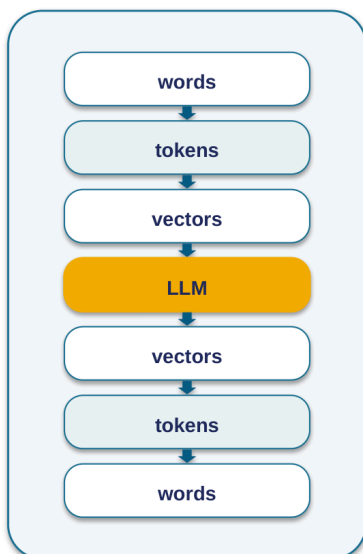
“similar” doesn’t mean
“about the same thing”



Similarity is a candidate-generation signal; relevance is a separate task: reranker, hybrid search, filters.

First — where similarity works well. Five short texts, a pairwise cosine similarity table on a modern embedding model: task-synonymous pairs — “configure SSL” and “installing an HTTPS certificate” — score around 0.85; topically related React texts — around 0.78; borscht against anything technical — 0.05 to 0.15. A query for “strawberries” finds documents about strawberry and wild strawberry without a synonym table — that’s what embeddings add on top of full-text search. Now — the boundary, and this is new material even for people already building semantic search. High cosine similarity means “about the same thing,” not “about the same thing with the same meaning.” The pair “how to configure SSL” and “how to disable SSL” gets a very high similarity score: same topic, same vocabulary, same syntactic frame — but the opposite practical meaning. An embedding averages the contextual statistics of an entire text; a short negation or an antonymic verb shifts the vector only weakly. In production this looks painful: a user asks how to enable certificate verification, search confidently surfaces an article on how to disable it, an LLM in a RAG pipeline dutifully summarizes it — and a system with great similarity metrics delivers harmful advice. The engineering answers are well known: a reranker — a second model that scores the full query-document pair; hybrid search combined with full-text search for exact terms; filters and metadata for directional attributes. And a diagnostic trick: assemble a test set of “query → correct document” pairs with deliberate traps — on/off, before/after, different versions — and see what ranks at the top. The full design of a RAG pipeline is Lecture 3; the principle to take from here is: cosine similarity is about topical closeness, relevance is a separate task.

Embeddings are the foundation of understanding: the model works with vectors, not strings



Paraphrases and synonyms

"How to set up SSL" and "Installing an HTTPS certificate" — close vectors → the model answers the same way; same with "car" and "automobile."

Cross-lingual proximity

"клубника" and strawberry — close vectors → the answer is correct regardless of the query language.

Sentence-level semantic proximity is the basis of "understanding" rephrasings.

What to do: one embedding model and one index serve search, clustering, and RAG all at once — but switching models means re-indexing the entire store; choose it as an infrastructure decision, not on the fly.

Choosing an embedding model for the task (MTEB, Matryoshka representations) — self-study material.

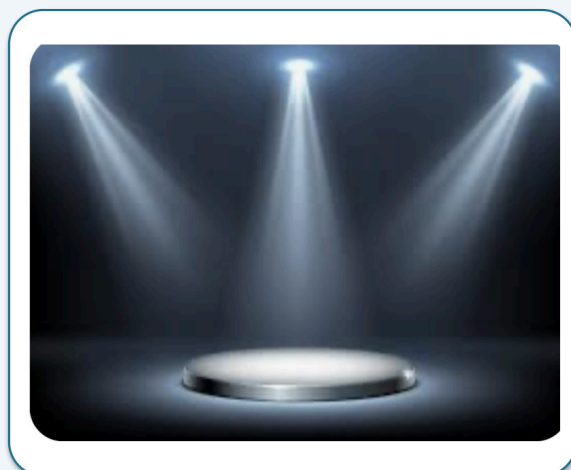
Let's tie the section together. The model doesn't work with words or with strings — it works with vectors; words only exist at the boundaries. On the input side, text passes through tokenization into identifiers, then through an input table into vectors; on the output side, an internal vector is turned back into text via a distribution and a token choice. Between human input and human output, the path is entirely vector-based. This explains what we intuitively call "understanding." When you ask "how to set up SSL" and then "installing an HTTPS certificate," the model answers similarly because both queries land at nearby points in the space. Same with synonyms and cross-lingual proximity: "клубника" and strawberry are close vectors, and the answer is correct regardless of the query language. The practical side: the same vectors underpin three standard applications — similarity search, unsupervised clustering, and semantic search with RAG. One embedding model and one index serve several product functions at once, which is why choosing an embedding model is an infrastructure decision, and switching is expensive: it means re-indexing the entire store. In production systems, semantic search almost never lives alone: the working standard is a hybrid of a full-text index for exact matches, a vector index for conceptual proximity, and a reranker on top — a direct consequence of the similarity boundary we just discussed. And remember: there's no inverse operation for an embedding — you can't recover text from a vector; it's a one-way compression of meaning. How to choose an embedding model for your language and dimensionality budget — I'll leave that to you for self-study: in the coursebook it's a chapter, part 1 — the section on 2026 embedding models.

Section 3

Attention Mechanism

“How the model decides what to rely on in the context — and what follows from that for roles, caching, and long windows”

4 case studies · 2 failures



Two stages of the pipeline are behind us. Tokenization: text is cut into tokens — identifiers from a vocabulary fixed before training. Embeddings: each identifier received a learned vector, and vector proximity reflects proximity of usage. The third stage is the attention mechanism, the richest part of the lecture. The input is a sequence of vectors; the stage's job is to decide, for each generation step, which parts of the context the model relies on most heavily. This mechanism directly gives rise to things you work with every day: why chat history gets cached and when the cache stops working; how prompt-caching economics work; why a prompt with a role genuinely changes the answer but doesn't make the answer more accurate; what the line "1 million token window" in a model card actually means — and where that promise breaks down. Order of movement: first the attention matrix and a precise definition with the three Query/Key/Value projections on our example; then KV-cache and the two generation phases; prompt-caching economics with a live case; then a worked example and the full mechanism of the role effect; the context-window race as of September 2026; and at the end of the section — an honest answer to the question of how much of that million-token window the model can actually use for reasoning.

Attention is a check of every token against every other token

Every token "looks at" every other token at once. At each step — $N \times N$ connections.

wt	The	cat	ate	the	mouse	because	it	was	hungry
The	1.0	0.3	0.2	0.1	0.1	0.1	0.1	0.0	0.0
cat	0.4	1.0	0.3	0.2	0.2	0.1	0.1	0.1	0.1
ate	0.3	0.4	1.0	0.2	0.5	0.1	0.1	0.1	0.1
the	0.1	0.1	0.1	1.0	0.3	0.1	0.1	0.0	0.0
mouse	0.2	0.2	0.4	0.3	1.0	0.1	0.1	0.0	0.1
because	0.1	0.2	0.2	0.1	0.1	1.0	0.3	0.2	0.2
it	0.1	0.7	0.1	0.1	0.1	0.2	1.0	0.3	0.4
was	0.1	0.4	0.1	0.1	0.1	0.2	0.4	1.0	0.5
hungry	0.1	0.4	0.1	0.1	0.1	0.2	0.4	0.5	1.0

In the row for "it": the largest weight lands on "cat," not "mouse" — resolved by semantic/thematic-role plausibility. A statistical association learned from the corpus.

In the decoder, a token sees only preceding tokens — the full matrix is shown here for clarity.

Dimensionality

$N \times N$, where N is the context length. Doubling the context **quadruples the attention compute**.

At every step

The weight distribution is recomputed from scratch at every generation step.

Multi-head

Each layer has dozens of parallel "heads" (typically 32–128); each captures its own type of relationship: grammar, semantics, long-range dependencies.

What this affects: the weights determine whose Value ends up in the current token's representation — and directly shape the next prediction.

You know the word "attention" from every other paper on transformers; let's pin down its exact form. Attention is not a linear but a matrix operation: every token in the context is checked against every other, and for a context of length N the weight map is N by N in size. This form immediately implies the single most important economic property of the whole architecture: doubling the context quadruples the volume of attention compute. When we later talk about the cost of million-token windows and why providers cache so aggressively, the root cause is exactly here. On the slide is a simplified seven-by-seven matrix for the sentence "The cat ate the mouse because it was hungry." The numbers are illustrative; real weights are normalized so that each row sums to one. Qualitatively the picture is typical: in the row for the token "it," the largest weight sits on the token "cat," not on "mouse" — the pronoun is resolved onto the more plausible bearer of the property "hungry": between an actor that ate and the thing that was eaten, the actor is the far more likely candidate for hunger as the motivating cause. This is what we colloquially call "the model understood what the pronoun refers to" — technically it's a statistical association learned from a huge corpus of plausible thematic roles: who tends to be hungry, who tends to do the eating. Two refinements to the picture. First: the weight distribution is recomputed from scratch at every generation step — this isn't "computed once and reused." Second: the real mechanism is multi-layer and multi-head — in each layer, dozens of "heads" work in parallel, typically 32 to 128, and each specializes in its own type of relationship: one captures grammatical agreement, another thematic proximity, a third long-range dependencies. Nobody designed these specializations — they emerged from training, because they help predict the next token. And the key point — what this weight distribution concretely affects, not just "something gets weighted somewhere." The weight of each connection determines what fraction of the corresponding token's Value vector ends up in the updated representation of the current position: if "it" assigns 0.7 of its weight to

"cat," then the content of "cat" — its Value — dominates the updated representation of the token "it." And the current token's representation is exactly what the probability distribution for the next prediction is built from at the next step. In other words: the attention weight distribution directly and mechanically determines what the next predicted token will be — this is not a side effect, it's a direct causal chain.

Attention returns a weight distribution: three projections, Query / Key / Value

Q is about the current step. K and V are about the already-processed context.

Query — “what I’m looking for right now”

Key — “what I offer”

Value — “what I hand over if I’m picked”

“The cat ate the mouse because **it** was hungry”

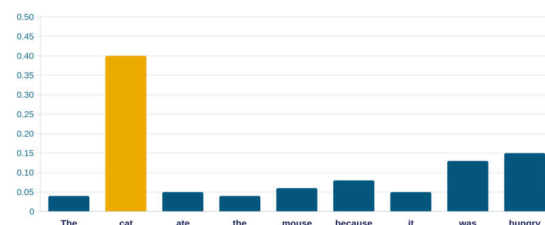
Q (“it”) = “looking for: who might have been hungry”

K (“cat”) = “I am an animate subject”

V (“cat”) = content that flows into “it”’s representation

A strong match between Q(“it”) and K(“cat”) gives a high weight → V(“cat”) determines the updated representation of the token “it.”

Weight distribution — sum = 1, leader “cat”



1. Input — all tokens in the context.

2. Output — a weight distribution, sum = 1.

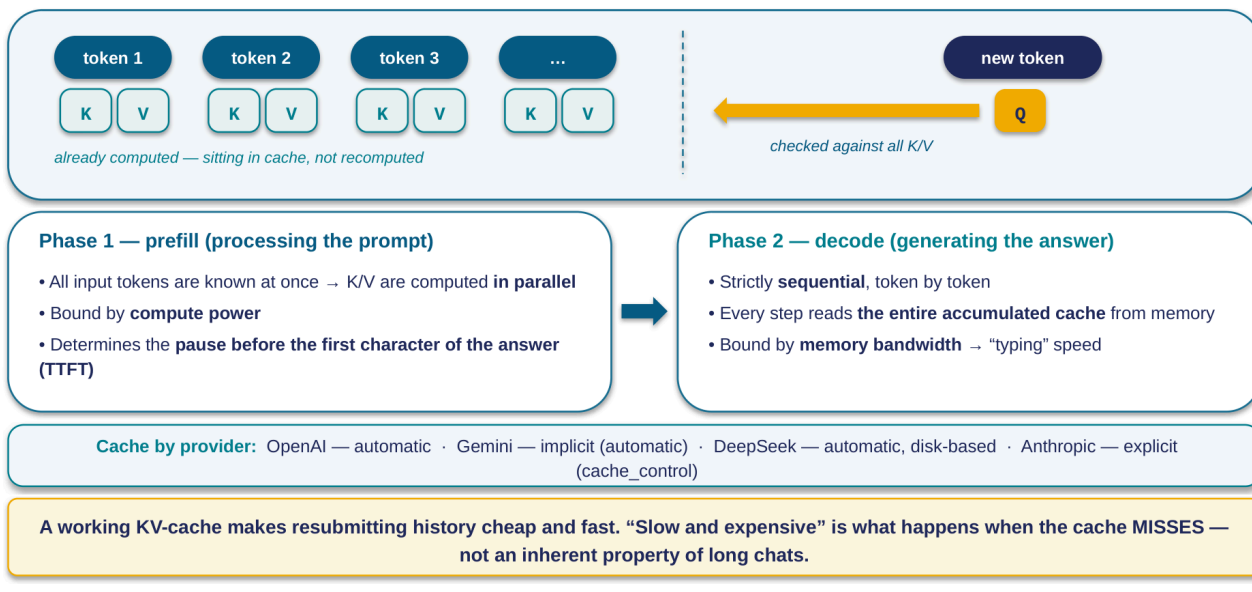
3. Recomputed at every generation step.

Metaphor: a flashlight in a dark room — the beam points at relevant tokens, brightness = attention weight.

Let's pin down the definition precisely. The working metaphor is a flashlight in a dark room: every token in the context is present, but the beam points at the ones relevant to the current question, and the brightness on each object is its weight. Formally, at each step attention returns a weight distribution over all tokens in the context, the sum equals one, and this distribution is recomputed from scratch at every generation step. Now let's go one level deeper, on the same sentence, so we don't lose the concrete picture. For every token, the model computes three projections of its vector: Query, Key, and Value. Take the token "it" from "The cat ate the mouse because it was hungry." Its Query can be roughly read as "looking for: who might have been hungry" — the current position's request to the rest of the context. The token "cat" offers its own Key — something like "I am an animate subject," a business card by which other tokens' queries find it. When the Query for "it" and the Key for "cat" match well, the weight between them comes out high — that's exactly what we saw in the matrix on the previous slide. Then the Value for "cat" comes into play — the content that actually flows into the updated representation of the token "it." The final representation at the position "it" is a weighted sum of the Values across the whole context, where the Value for "cat" contributes the most. In plain terms: Query is the question a token asks; Key is what a token uses to answer other tokens' questions; Value is what a token actually hands over if it's chosen. One structural observation is worth remembering: Query is about the current step, while Key and Value are about the already-processed context, which doesn't change. This asymmetry is what gives rise to the single biggest optimization in the entire model-serving industry — we'll see it on the next slide, when we look at why a long chat slows down.

K and V are cached — only Q is recomputed

KV-cache: the Key/Value of already-processed tokens are stored in accelerator memory. At each step, only the Q of the new token is computed — against the stored K/V.



Recall the observation from the previous slide: Query is about the current step, Key and Value are about the already-processed context, which doesn't change. Generation is autoregressive: tokens come out one at a time, and a naive implementation would recompute the K and V of every token from scratch at every step — but they don't change. Hence the central optimization of all large-model inference — KV-cache: the Key and Value vectors of already-computed tokens are stored in accelerator memory, and at each step only the Q of the new token is computed, along with its products against the stored K/V. Out of the cache grows the two-phase asymmetry you observe in every request. Prefill — processing the input prompt: all of its tokens are known at once, so their K/V are computed in parallel in one large matrix computation; this phase is compute-bound and determines the pause before the first character of the answer. Decode — generation: strictly sequential, and at every step you have to read the entire accumulated cache from memory; this phase is bound by memory bandwidth and determines the speed in tokens per second. That's why a long prompt mainly stretches out the pause before the first token, while a long context overall slows down every generation step. The exact wording of the conclusion matters here, because it's easily distorted into the folk wisdom "new task → new chat." As long as the KV-cache works — that is, the history matches what's already been computed — resubmitting the history is cheap and fast: that's the whole point of the cache's existence. "Slow and expensive" doesn't happen as a property of long chats in general; it happens specifically when the cache misses — when the context changes at the beginning, when the session has expired, when the provider has evicted your cache from memory to serve someone else's load. The history is submitted to the model in full on every turn — the model is stateless, as we established in Lecture 1 — but that doesn't mean every turn is recomputed from zero. Cache implementation differs across providers, and this directly affects what you need to do by hand. At OpenAI and DeepSeek the cache kicks in

automatically — no special setup in the request; at DeepSeek it's also disk-based, not just in accelerator memory. Google Gemini has implicit caching, also on by default. Anthropic's cache is explicit: you need to place the ``cache_control`` marker at the right spots in the prompt yourself — otherwise the cache won't trigger, even if the content matches. Practical takeaway: if you're on Anthropic and not seeing savings, check whether the marker is placed where it needs to be; on the other three, check the cache-hit-rate metrics in the API response, because "automatic" doesn't mean "guaranteed to trigger." Scale of the effect: at contexts of hundreds of thousands of tokens, one user's cache occupies gigabytes of accelerator memory — hence providers' aggressive batching and the premium pricing on long contexts.

Prompt caching is a bet on reuse, not a discount

Three requests in a row:

Request 1: the prefix is written to the cache — write costs 1.25–2× the rate

Request 2: same prefix → cache hit — reads cost 0.1× (newest models — 0.025×)

Request 3: a line was added at the front → prefix no longer matches → cache miss, full price

Case: 50,000 document analyses per month

without cache \$45,000

\$8,000 with cache · -82%



Condition: exact prefix match. One changed token invalidates the cache for everything after it.

system prompt

✓ cache

instructions

✓ cache

documents

✓ cache

"Today is 2026-09-05 14:23" — dynamic

!

question

× cache miss

A dynamic element at the start (timestamp) breaks the cache for every block after it — this is "request 3."

Composition rule: stable content goes first (prompt, instructions, examples, documents), variable content goes last.

KV-cache lives inside a single generation. Providers took the next step: if two different requests share the same prefix — the same system prompt, instructions, documents — their K/V can be reused across requests. This is prompt caching, and since 2025–2026 it's the main lever for optimizing the cost of LLM workloads. Let's trace the mechanics through three sequential requests — it's clearer this way than through isolated numbers. First request: you send the prompt for the first time, there's no cache yet — the prefix is written to the cache, and writing costs more than a normal input, anywhere from 1.25 to 2 times the rate depending on the cache's time-to-live. Second request: you send the same prefix — say, the same system prompt and the same set of documents, only a new user question at the end — the prefix matches byte-for-byte, the cache hits, and you pay a tenth of the base input rate, on the newest models as low as a fortieth. Third request: someone added one line at the start of the prompt — say, the current date or an updated instruction — and now the prefix no longer matches what's in the cache starting from that line. Cache miss: the entire load is billed at full price, as if the cache had never existed, and if you're using explicit writes, you also pay the write surcharge on top. The cache isn't free — it's a bet on reuse: it pays off from the second or third hit onward, while a workload made of unique requests with no shared prefix only gets more expensive. The scale of the effect is real: a case of 50,000 document analyses per month — \$45,000 without caching versus \$8,000 with caching, an 82 percent saving. The main technical condition is visible in the third request — an exact prefix match: the cache only hits if everything up to the checkpoint matches byte-for-byte. A classic self-inflicted wound is adding something variable to the start of the system prompt, like the current date and time: then no request will ever land in the cache. Hence the composition rule: stable content first, variable content last. And a one-minute audit for a running pipeline: look at the cache-accounting fields in the API responses — if cache reads are zero despite a stable system prompt, you can almost

certainly fix this in one evening by reordering the prompt. For multi-step agents, the cache isn't an optimization — it's a condition for profitability: we'll come back to that in the next lecture.

A role works through attention weight — but doesn't raise factual quality

Worked example: where does "it" look?

"The cat ate the mouse, because it was hungry"

main weight

Simplification: an aggregate of hundreds of connections across dozens of layers — the model reproduces correlations of usage, not grammatical parsing.

Think for 30 seconds: where will the weight from "it" go in "The server crashed because it ran out of memory"?

No role

"Explain the GIL" → a neutral, generic answer.

With a role

"You are an experienced Python developer. Explain the GIL" — role tokens get weight → shift the distribution of subsequent tokens: more specific, expert register.

Zheng et al. (2024, EMNLP Findings; 2410 questions, 162 personas): a persona/role in the prompt does not raise factual accuracy — the effect of any specific role is unpredictable.

Separately from that study — from course observations: a role noticeably changes the tone, style, and content selection of the answer.

A role is a tool for controlling style and focus, not an "intelligence booster." If you need an answer grounded in your data, give it the data — not a third adjective attached to the word "expert."

Warm-up: "The cat ate the mouse because it was hungry." Reaching the token "it," the model has to resolve what it refers to — and in the attention map the weight is distributed in favor of "cat." The three arrows are a simplification: an aggregate of hundreds of connections across dozens of layers; the model isn't doing grammatical parsing, it's reproducing correlations of usage. Test your intuition: "The server crashed because it ran out of memory." On most models the weight will go to the nearest plausible entity given the semantic context — if your intuition gave a similar answer, you're already thinking in terms of corpus statistics rather than grammar. Note the shift from the previous example: there, the cue was thematic-role plausibility (who is the likely bearer of "hungry"); here, there's no animate-subject cue at all — "it" plausibly refers back to "server," the only entity that can "run out of memory," which is itself a giveaway of the same underlying mechanism: attention resolves pronouns via learned plausibility of who-does-what, not via a fixed grammatical rule. Now — the reason you already know all this. Let's compare two prompts that are essentially the same. "Explain the GIL" with no role — the model answers neutrally, generically. "You are an experienced Python developer. Explain the GIL" — you've surely noticed that a prompt like this works differently. The mechanism: the tokens "experienced," "Python developer" receive weight in the attention distribution during the generation of every answer token, and the choice shifts toward whatever is consistent with them — the answer becomes more specific and more confident, in an expert register. A role isn't a request to "trust me" — it's an input signal that physically participates in the weighting. And the boundary of the effect, important to know before you start overusing roles in your prompts: Zheng et al. (2024, EMNLP Findings) tested 2410 questions and 162 personas — a persona/role in the prompt does not raise factual accuracy, and the effect of any specific role is unpredictable. Separately from that study: from course observations, a role noticeably changes the tone, style, and content selection of the

answer — a distinct effect, not the same thing as accuracy. A role shifts the distribution, it doesn't add knowledge. If the model doesn't know the answer, the role "expert" won't make it know — it will make it sound more confident without being more correct.

Practical takeaway: a role is a legitimate, useful tool for controlling the style and focus of the answer, but don't confuse it with raising accuracy. If you need a precise answer grounded in your data, give the model the data — not a third adjective attached to the word "expert."

The 2026 frontier standard is a window of up to 1 million tokens. But advertised ≠ usable

GPT-3.5 (2022) — 4K



Claude 3.5 (2024) — 200K



Frontier standard (2026) — up to 1M · Fable 5, GPT-5.6, Gemini 3.1 Pro and others



Gemini 3.5 Pro (2026) — 2M · a single outlier model, not the standard



width — logarithmic scale

Llama 4 Scout: "10M"

advertised; no published benchmark confirms quality near the limit

YandexGPT 5 Pro: 32K

one to two orders of magnitude below the flagships
— for long documents this is a defining constraint

You can't just "stretch" the window: token position is encoded by a geometry trained on specific lengths — extending it (RoPE / YaRN) is separate engineering work.

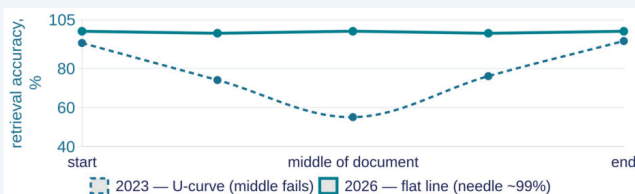
You pay for what you put in the window, not for what the window can hold: 900K input tokens at \$10/million ≈ \$9 for a single call.

What to do: choose a model by the task's effective window (benchmarks without lexical shortcuts), not by the marketing maximum.

The context window — the maximum number of tokens per request — has grown three orders of magnitude in four years: 4,000 for GPT-3.5 at the launch of ChatGPT, 200,000 for Claude 3.5 in 2024, and the 2026 frontier standard — up to one million: Fable 5, GPT-5.6, Gemini 3.1 Pro and others hold this level, and for some of these models the full window is included in the standard price with no length surcharge. Two million for Gemini 3.5 Pro is not the standard but an outlier — for now the only model at this level, not a typical flagship; there are already more than a dozen models on the market with windows up to a million. Two sobering outliers around the standard. On the high side — marketing: Llama 4 Scout advertises ten million tokens, but no published benchmark confirms that quality holds up anywhere near that limit; "advertised window" and "usable window" are different quantities, and we'll measure the gap between them a couple of slides from now. On the low side — a contrast: YandexGPT 5 Pro works with a 32,000-token window — for tasks with long documents, that's not a nuance, it's a defining constraint on model choice. Why is the window finite, and why can't you "just increase it"? The first reason is familiar: the quadratic cost of attention plus a linearly growing cache. The second is subtler: token position is encoded in the model's geometry in a way trained on specific lengths, and naively stretching it breaks the mechanism; extension methods — RoPE, YaRN — are covered in the coursebook for self-study: the chapter, part 2 — the section on positional encoding. And the money arithmetic that's worth doing once. The full window is tokens you pay for as input on every request: a call to a premium model at \$10 per million input tokens, filled to 900,000, costs around \$9 — for a single call; ten turns of dialogue run close to a hundred dollars without caching. The question "how much context does this task need" matters more economically than the question "how much can the model accept."

Verbatim retrieval is nearly solved. Understanding long context is not

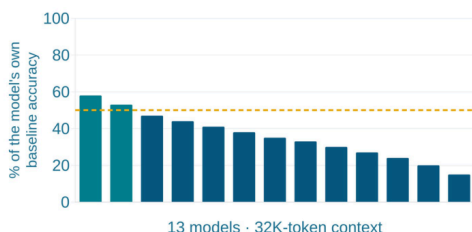
Forgetting is solved ✓ — finding a verbatim insertion (needle-in-a-haystack)



The 2023 U-curve has flattened: **single-needle — up to 99% on the full 1M window** (Gemini Deep Think).



Understanding by meaning — no (NoLiMa, 2025: the benchmark removed literal word overlap)



Without lexical overlap, quality falls **below 50% of the baseline already at 32K — for 11 of 13 models.**

32K tokens ≈ 3% of a flagship's advertised window; the baseline is the same model's accuracy on short context.

A 1M window ≠ 1M tokens of reasoning. The window is how much the model can read; the usable length is how many tokens out it can still connect facts across.

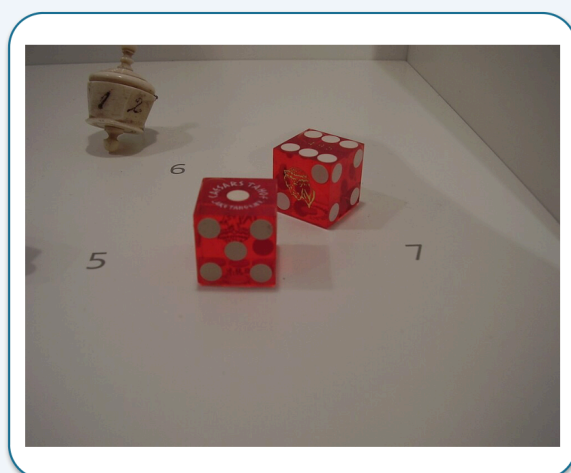
You probably know the classic 2023 result "Lost in the Middle": a fact buried in the middle of a long context is retrieved worse than one at the edges — a U-shaped accuracy curve. That knowledge needs a 2026 update — in both directions. The good news: literal "needle in a haystack" search — finding a verbatim inserted phrase — is essentially solved by the flagships: up to 99% on the full million-token window. If the task is to find where the contract mentions the amount, a large window works almost as advertised. The bad news came from the NoLiMa benchmark, which removed the main crutch of such tests — literal lexical overlap between the question and the hidden fragment. When you have to find something by meaning rather than by matching words, the picture collapses: eleven of thirteen tested models fall below half of their own accuracy on short context — note the baseline: it's the model compared against itself, not some absolute threshold. And this happens already at 32,000 tokens — not at a million, but at three percent of a flagship's advertised window. The U-curve didn't disappear — it hid behind high scores on tests that measure retrieval, not reasoning. A formula worth remembering: a million-token window doesn't equal a million tokens of reasoning. The window is how much the model can read; the usable length is how many tokens out it can still connect facts across without literal cues, and the second quantity is many times smaller than the first. Consequences: put critical instructions at the start or the end, not in the middle; "dump the whole knowledge base into the window" loses to good retrieval of five to ten targeted fragments — more on that next lecture; and when choosing a model for long documents, don't ask "what window size" but "what results on benchmarks without lexical overlap" — and test at your task's actual working length: half an hour of stress-testing on real documents tells you more than any leaderboard.

Section 4

Sampling and Generation

"How a probability distribution gives birth to a single token — and which knobs steer that choice"

4 case studies · 2 failures

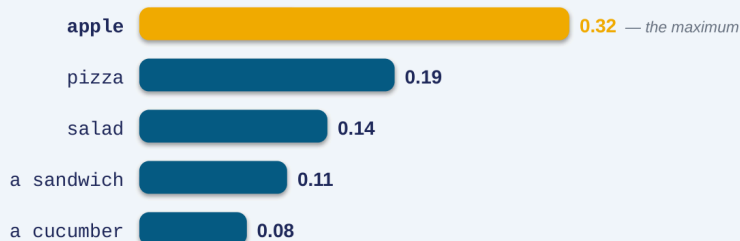


Three pipeline stages are behind us: text was cut into tokens, tokens received vectors, attention decided what to lean on in the context. The result of those three stages is a probability distribution over the model's entire vocabulary: for each of roughly two hundred thousand tokens, a probability of being next. The fourth stage is sampling: the rule for picking one token out of that distribution. All the "creativity" and all the "randomness" of large models live in this rule, and it is the only part of the pipeline you control directly, through the API's knobs. The order we'll move through the section. First we'll pin down the distribution itself — that it is the "real" output of the model, and everything else is a policy layered on top of it. Then the precise mechanics of temperature and its neighbors — with a live comparison of behavior at zero versus one-point-five. Next — a check of the trickiest of the six claims: does zero temperature really give identical answers every time; the answer will likely surprise you, and it matters for anyone writing tests against an LLM. After that — how the classic set of four API knobs grew to six, guaranteed response format via structured outputs, the autoregressive loop that assembles everything into one whole, and reasoning tokens — an invisible but billed line item on your invoice. We'll close the section with a fresh look at the choice between a local model and the cloud.

At every step — a probability distribution over the entire token vocabulary

Context: "Today I ate ..."

$P(\text{next token})$:



The remaining ~200,000 vocabulary tokens — each below 0.05. All probabilities sum to 1. The numbers are illustrative.

Sampling → one token

the rule for picking one token from the distribution — the only knob in your hands



[apple]

one is chosen — the remaining candidates vanish from the answer

The distribution is the model's "real" output. A confident answer and a hallucination existed simultaneously before sampling — as probability mass; the policy made the choice.

The result of three pipeline stages: the model's output is a probability distribution over the entire vocabulary — for each of roughly two hundred thousand tokens, a probability of being next; the sum equals one. For the prompt "Today I ate," the distribution looks roughly like this: "apple" — 0.32, "pizza" — 0.19, "salad" — 0.14, then a long tail; the numbers are illustrative, but the picture is stable. The distribution isn't uniform — the model has statistical preferences — but it isn't a single spike either: there are several plausible candidates. Sampling is the rule for picking one token out of this distribution. All the "creativity" and all the "randomness" of large models live in this rule, and it is the only part of the pipeline you control directly, through the API's knobs. It's worth internalizing once and for all how much this distribution is the "real" output of the model, with everything else a policy layered on top of it. A confident answer and an evasive one, an accurate fact and a hallucination, "Paris" and "perhaps you mean..." — before sampling, all of this existed simultaneously, as probability mass spread across different continuations; the policy made the choice, not some "opinion" of the model. A practical consequence for anyone building pipelines: some providers expose a slice of this information through the API — log-probabilities of the top candidates at each step. This is a useful diagnostic channel: a wide, spread-out distribution at a key token in the answer is an honest signal of the model's uncertainty, one that the answer's own text may never reveal. For classification, this is a cheap way to get a confidence measure without a second call asking "how sure are you?" — that second call, incidentally, doesn't measure the model's confidence, it measures how well-trained it is at answering such questions.

Temperature is a divisor on the logits: sharpness of choice, not knowledge

$T = 0 \Rightarrow \text{choice} = \text{argmax } P(\text{token})$

$T > 0 \Rightarrow \text{sampling from a } P^{(1/T)}\text{-shaped distribution}$

logits are divided by T before softmax: $T < 1$ sharpens the distribution, $T > 1$ flattens it

$T \rightarrow 0$ (argmax)



Picks the most likely token. Nearly identical answers — we'll unpack that "nearly" on the next slide.

$T = 1$ (default)



Sampling proportional to the model's probabilities.

$T = 1.5$ (smoothing)



Rare tokens get a real chance — anywhere up to incoherence.

Why T matters even though the ranking of candidates doesn't change: the choice isn't "take the top-1" but a random draw proportional to probabilities; T reshapes the probabilities themselves.

top-p — cuts the tail by probability mass · **top-k** — by number of candidates · T per task: 0–0.3 code and classification, 0.7+ generation.

Live run: one prompt — 10 times at $T=0$ and 10 times at $T=1.5$.

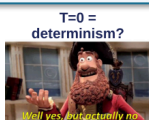
You already know these knobs — let's sharpen the mechanics into an exact formula. At $T=0$, the choice is $\text{argmax } P(\text{token})$: deterministically take the token with maximum probability, with no sampling at all. At $T>0$, the model samples from a distribution transformed by the rule P to the power of $1/T$ — technically this means the raw logits are divided by T before passing through softmax and turning into probabilities. It's worth pausing on a point that often causes confusion: token ranking by probability doesn't change with temperature — the most probable token stays the most probable at any T . So why does temperature matter at all if the ranking is the same? The answer is that choosing a token isn't "take the top-1 off a list" — it's a random draw where the probability of picking each token is proportional to its probability in the distribution. And temperature changes those probabilities themselves, not their order: at T below one, the distribution sharpens — the leader gets even more probability mass, the outsiders lose their chances down to nearly zero; at T above one, the distribution flattens — the gap between the leader and the rest shrinks, and tokens that were previously almost impossible get a real chance of being picked. That's exactly why $T=0$ is the limiting case: the sharpening is taken to its extreme, and the leader's probability becomes exactly one — argmax . Second tier: top-p cuts the tail by probability mass — we sample from the smallest set of the most probable tokens whose combined probability is at least p ; top-k cuts by number of candidates. Practice hasn't changed: the main knob is temperature, these two are fine-tuning. Now the live run: the same prompt — "come up with a name for a note-taking service" — ten times at zero and ten times at one-point-five. The first row gives nearly identical answers — "Notely," "Notely," "Notely," with maybe an ending changing here and there; the word "nearly" carries more weight here than it seems, and the next slide is devoted to it. The second row spreads from lucky finds to gibberish: "MindStream," "dawn-fog notebook," "breath-of-numbers pad" (examples are illustrative — you'll see your own spread of

variety in a minute). The value of this experiment, if you run it yourself, isn't confirming the theory — it's calibrating your hand: you'll see what the tail of the distribution looks like for your specific task and at what temperature it starts leaking into the answers. And the reverse exercise: if your prompts in production run at the provider's default temperature — find out what that default actually is; defaults differ across providers and change over time, and a deliberate choice here costs one line of code.

T=0 does not give determinism: 80 unique answers out of 1000

80 / 1000

unique response variants to an identical request at T=0 —
stock vLLM (an open inference server; Thinking Machines
Lab, September 2025)



The cause isn't "floating-point in general" but a lack of **batch invariance in the kernels**:

1. The server groups concurrent requests from different users into batches
2. Batch size depends on other users' load in that exact millisecond
3. Different batch size → different summation order → different least-significant bits
4. Two close argmax candidates → the least-significant bit decides the token → autoregression spreads the divergence through the whole answer

A fix exists: batch-invariant kernels — 1000/1000 bit-for-bit identical

Cost: ~35% of throughput → providers don't turn it on; OpenAI's seed is officially "mostly deterministic," not fully

You cannot get a guaranteed-deterministic answer from a cloud LLM today — build your processes accounting for that.
Consequence: don't compare answers bit-for-bit, compare them semantically or structurally.

This is the trickiest of the six claims, because it's almost true, and tests and pipelines get built on it. You set temperature to zero, expecting: same request, same answer. A check on real infrastructure: stock vLLM, a thousand runs of an identical request — eighty unique answer variants. Zero really does make the argmax choice deterministic — but the distribution itself, the one argmax is taken from, turns out to be slightly different from run to run. The cause runs deeper than the common "floating point on the GPU" explanation. The main culprit is a lack of batch invariance in the compute kernels. A provider's server dynamically groups concurrent requests from different users into batches; batch size depends on the load at that exact millisecond; and many kernels use a different summation order depending on batch size. Floating-point addition is non-associative — order changes the least-significant bits — and when two argmax candidates are close, the least-significant bit decides the token choice; from there, autoregression spreads the divergence through the entire answer. Your "deterministic" request is non-deterministic because you're sharing the server with other users, and their traffic changes your batch size. The most instructive part: the problem is solvable — and the fix has been rejected on economic grounds. The same authors wrote batch-invariant kernels: a thousand out of a thousand runs, bit-for-bit identical. The cost is roughly 35 percent of throughput, which is why providers don't turn this mode on by default; hence the status of OpenAI's seed parameter — officially "mostly deterministic," with no guarantees. The main takeaway is worded this way, and the order matters here: getting a guaranteed-deterministic answer from a cloud LLM is not possible today — this isn't a temporary bug, it's a consequence of providers' economic choice, and you need to design your processes with that fact in mind, not in the hope that "the next version will fix it." From this, as a consequence rather than the main thesis: don't build tests on bit-for-bit comparison of answers — compare them semantically or structurally; pin down reproducibility with a dataset and a metric. And if

strict determinism is genuinely required — for an audit, for regulatory reasons — that's a separate infrastructure requirement with a price tag of a third of your throughput, budgeted for from the start.

API knobs: reasoning depth has joined randomness and length

Parameter	Range	What it affects	Typical value
temperature	0–2	Determinism ↔ chaos of token choice	0 classification; 0.7–1.2 text
top_p	0.1–1	Width of the candidate tail during sampling	0.9–0.95
max_tokens	integer	Hard cutoff on generation — can stop mid-way	set per task, with headroom for JSON/code
effort	none → xhigh	Depth of internal reasoning — and its cost	medium by default for most providers
verbosity	low → high	Length of the visible answer, independent of thinking depth	medium

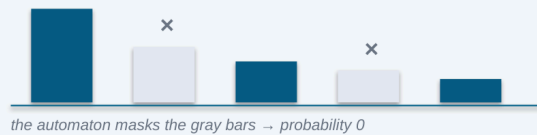
effort / reasoning_effort — a new axis for 2026: OpenAI — an effort scale; Anthropic — adaptive thinking; Gemini — thinking budget.

What to do: start tuning with temperature and effort — these are the two main knobs; top_p / top_k / verbosity are fine-tuning on top.

The classic set — temperature, top_p, max_tokens — you already know; the parameter table on the slide pins down the range and typical value for each: zero for classification, 0.2–0.3 for code, around 0.7 for explanations, 0.9–1.2 for creative text. The 2026 news is that reasoning models have grown two new knobs, and they control a different axis. Effort, or reasoning_effort, is the depth of internal reasoning: OpenAI has a scale from "none" to "xhigh," Anthropic has an effort parameter under adaptive thinking, Gemini has a thinking budget, where zero turns reasoning off and minus one hands the choice to the model. Verbosity is the length of the visible answer, independent of reasoning depth: you can ask the model to think deeply but answer briefly. The old pair "temperature controls randomness, max_tokens controls length" has been joined by the pair "effort controls thinking, verbosity controls wordiness." A live example of API evolution — useful as an inoculation against memorizing parameters: in 2026, Anthropic broke backward compatibility for controlling thinking — the manual budget_tokens parameter now returns a 400 error on new models; in its place is adaptive thinking, where the model decides the depth itself. A detail at the intersection with caching: changing the thinking configuration between requests invalidates the prompt cache — it's part of the cacheable prefix. And a related consequence: don't carry habits between providers and generations. Knobs with identical names behave differently, and recommendations flip sign: OpenAI explicitly recommends against giving reasoning models prompts in the style of "let's think step by step" — the model already reasons on its own, and manually padding out the chain just duplicates work and cost. A technique that was best practice for years has become an anti-pattern. When you switch models, read the parameters page as carefully as you'd read a library's breaking-changes list for a major upgrade.

Structured outputs: invalid tokens are zeroed out right in the distribution

Constrained decoding: the JSON schema is compiled into a **finite-state machine over tokens**. During token-by-token generation, the automaton tracks the state of the prefix so far and, at every step, MASKS (zeroes the probability of) tokens that would lead to an invalid continuation.



Asking "answer strictly in JSON" → ~80% valid

Strict mode → **100%** — the output is valid by construction, not "checked after the fact"

Limitations — properties of compilation

Recursion via \$ref — not supported (tree → flat list with parent_id)

Nesting depth — ≤ 5

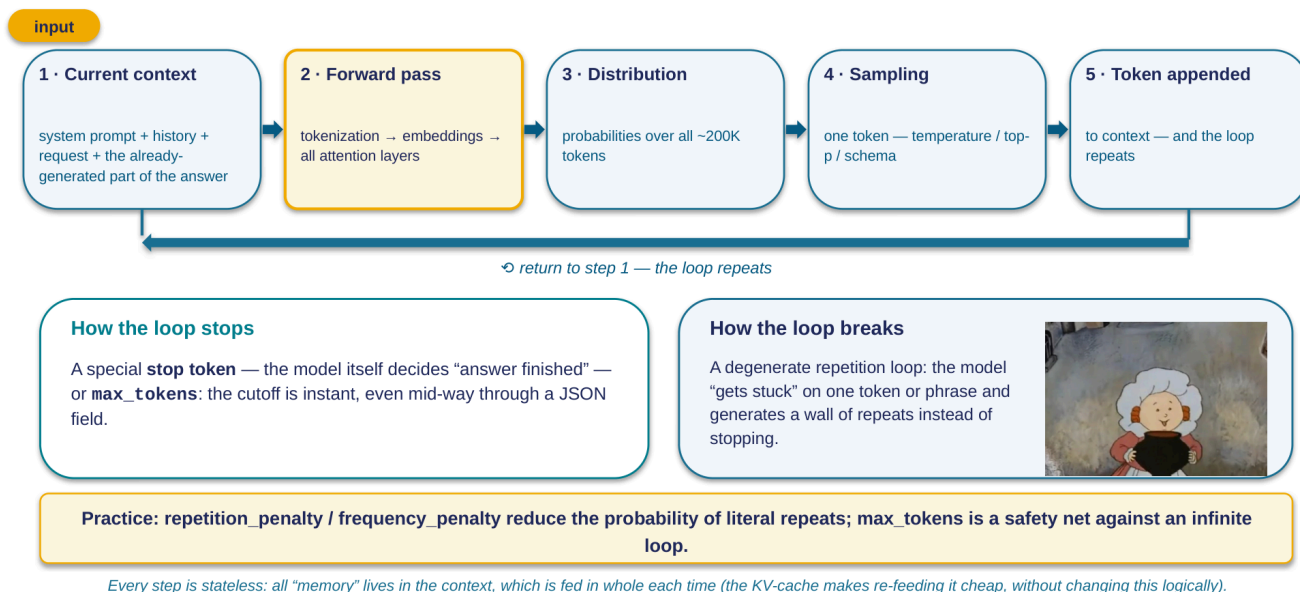
The first request with a new schema pays for grammar compilation — up to 10 s

Guarantees syntax, not meaning: validating values is still your job

Ask the room: why exactly 100%, and not 99.9?



Anyone who has ever asked a model to "answer strictly in JSON" knows the price of the word "strictly": an ordinary request produces valid JSON about eighty percent of the time, and the remaining twenty percent breaks the pipeline. Structured outputs solve this not through persuasion but through mechanics: the given schema is compiled into a finite-state machine over tokens; during token-by-token generation, the automaton tracks the state of the already-generated prefix and, at every step, masks — zeroes the probability of — tokens that would lead to an invalid continuation, right inside the very distribution we examined at the start of the section. That's why the output is valid by construction, not "checked after the fact." A question for the room, thirty seconds: why is it exactly one hundred percent, not 99.9? The answer: because the guarantee is built into the sampling itself — the model is physically unable to pick a token that violates the schema; this isn't a post-hoc check with a retry, it's a filter applied at the moment of choice. OpenAI calls this strict mode, Anthropic has native output_format, Gemini has response_schema. Understanding the mechanism immediately explains the limitations — they aren't API quirks, they're properties of compiling a grammar. The schema has to compile into an automaton over tokens: recursion via references isn't supported — a genuine unbounded-depth tree can't be expressed as a finite grammar, so nested structures are modeled as a flat list with a parent field; nesting depth is capped at five levels; the first request with a new schema pays for compilation — typically up to ten seconds. So don't generate schemas dynamically on every request — fix and version them like code. And one last boundary: syntax is guaranteed, not the meaningfulness of the fields — validating values is still your job. A design recommendation: start with a schema simpler than you'd like — flat structures work reliably and cheaply, and every extra level of nesting brings you closer to the compiler's limits. A schema is an interface contract with a non-deterministic executor; the shorter the contract, the fewer ways there are to break it.

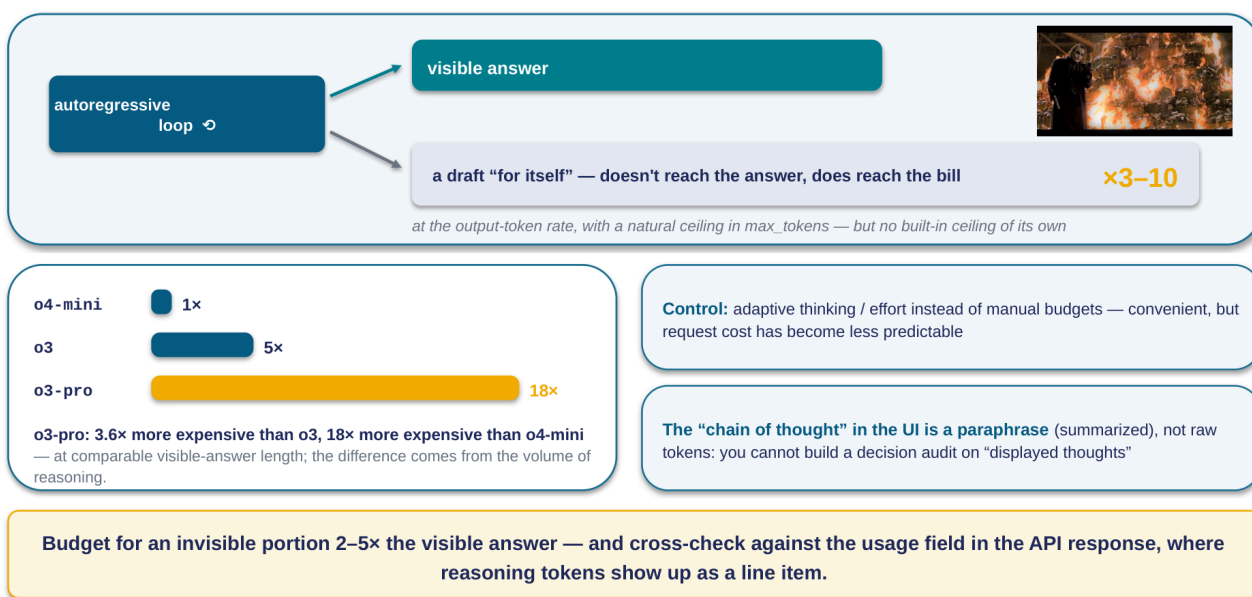
Predict a token → append it to context → predict the next one

Let's assemble the pipeline into a loop — this is the core of the whole picture.

Autoregressive generation: the current context — the system prompt, history, request, and the already-generated part of the answer — goes through a forward pass through tokenization, embeddings, and all the attention layers; the output is a probability distribution over the next token; sampling picks one token; the token is appended to the context; the loop repeats until a special stop token or the max_tokens limit. Every step is stateless: all “memory” lives in the context, which is fed in whole — with an adjustment for the KV-cache, which makes re-feeding it cheap without changing this logically. The answer being typed out in front of you isn't for show — it's the physical pace: one token per pass. The loop can stop correctly — or it can break. Correct stopping is a special stop token or the max_tokens limit; a cutoff by limit is instant, even mid-way through a JSON field, and post-processing has to catch that case. A break looks like a degenerate repetition loop: the model gets stuck on one token or a short phrase and generates a wall of repeats instead of a meaningful continuation — the “Pot, stop boiling!” folk tale (a Russian telling of the classic magic-porridge-pot story — the cauldron that won't stop without the right stop-word), except here the pot is the model and the porridge is the tokens. [Space for the lecturer's own story: if you've had a real case where a model fell into this kind of loop in production or testing, tell it here in your own words.] The mechanism is the same loop: if the distribution at some step is heavily skewed toward text already generated (say, because of low temperature or prompt structure), the model repeatedly picks a similar token, and autoregression locks in the pattern. In practice: repetition_penalty and frequency_penalty reduce the probability of literal repeats of already-generated tokens; max_tokens remains a safety net regardless — even if the model gets stuck, generation won't run forever. An end-to-end trace — let's assemble the whole pipeline on one example. You send “The capital of France is.” Tokenization: the text is cut into three or four tokens by a table fixed before

training. Embeddings: each ID pulls its own vector from the input table. Attention: at the position after the dash, Query "asks," and Key and Value for "capital" and "France" answer with weights — and by the way, they're already sitting in the cache and don't get recomputed. Output: a distribution where "Paris" gets, say, 0.93. Sampling: at zero temperature we take the maximum — "Paris"; the loop turns one more time and picks the stop token. If reasoning mode had been on, the loop would have wound through draft tokens before "Paris"; if a JSON schema had been set, the distribution at every step would have been filtered by the grammar. One request — every stage, every knob. Course terminology norm: "autoregressive," not "auto-regressive."

Reasoning tokens are invisible but billed as output



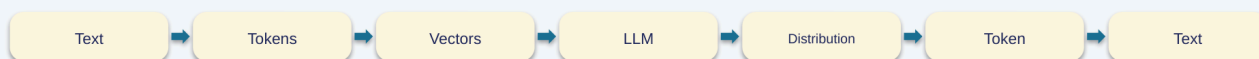
Reasoning models — OpenAI's o-series, Claude's extended thinking, Gemini's Deep Think — don't change the loop we just assembled. What they do is different: before the visible answer, the model uses that same autoregressive loop to generate reasoning tokens — a draft "for itself" that never makes it into the answer. The claim "if you can't see it, it isn't billed" is closed off by a single line from the billing docs: reasoning tokens are billed as output tokens, at the most expensive rate, and they count against the max_tokens limit. The scale is worth feeling in numbers. The volume of invisible reasoning inflates three to ten times relative to the visible answer, with no natural ceiling. In a typical agentic workload, o3-pro at maximum reasoning depth ran to roughly 280 dollars a month — three-point-six times more expensive than o3 and eighteen times more expensive than o4-mini — even though the visible answers of all three are comparable in length; the difference comes entirely from the volume of reasoning. When budgeting, plan for an invisible portion two to five times the visible answer — and double-check against the usage field in the API response, where these tokens appear as a line item. Two more boundaries. First: what you see as the "chain of thought" in the interface is a paraphrase — providers give you summarized thinking by default, a separately generated text, not raw tokens; you cannot build a decision audit on it. Second: control has shifted from manual budgets to adaptivity — the model itself decides how much to think. This is convenient — and it also means request cost has become less predictable: for bulk processing, push effort down explicitly, and turn on the "thinking" mode only where the complexity justifies it. The default depth is often excessive: if you haven't measured how quality depends on effort for your own task, odds are you're overpaying by a factor of two.

Section 5

Model Types and Sizes

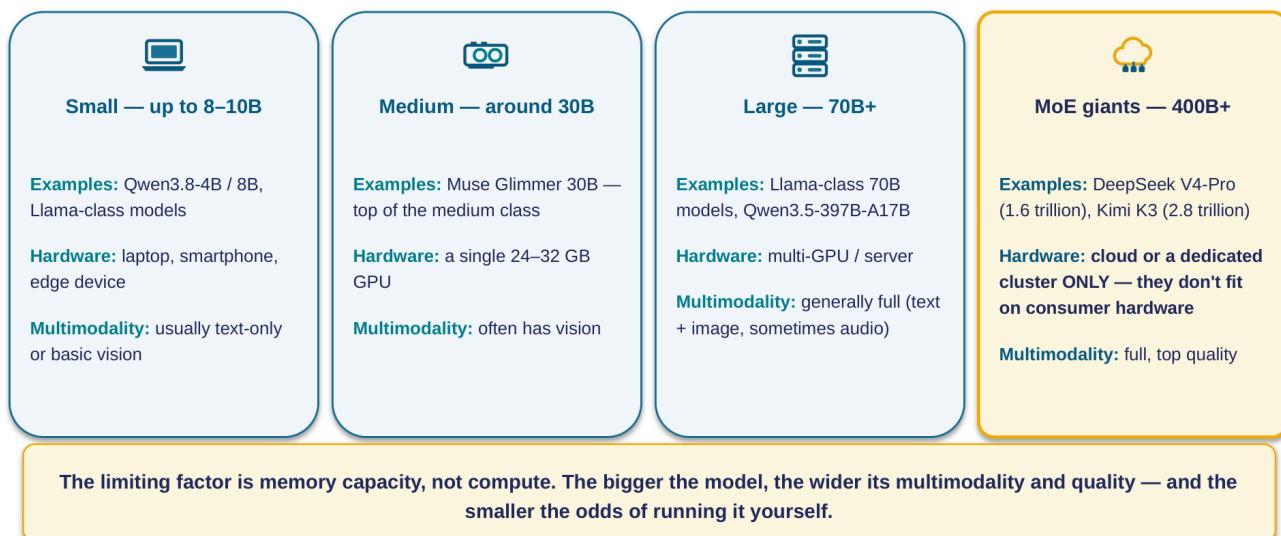
“What models run on, of different sizes — and what each class can do”

4 case studies



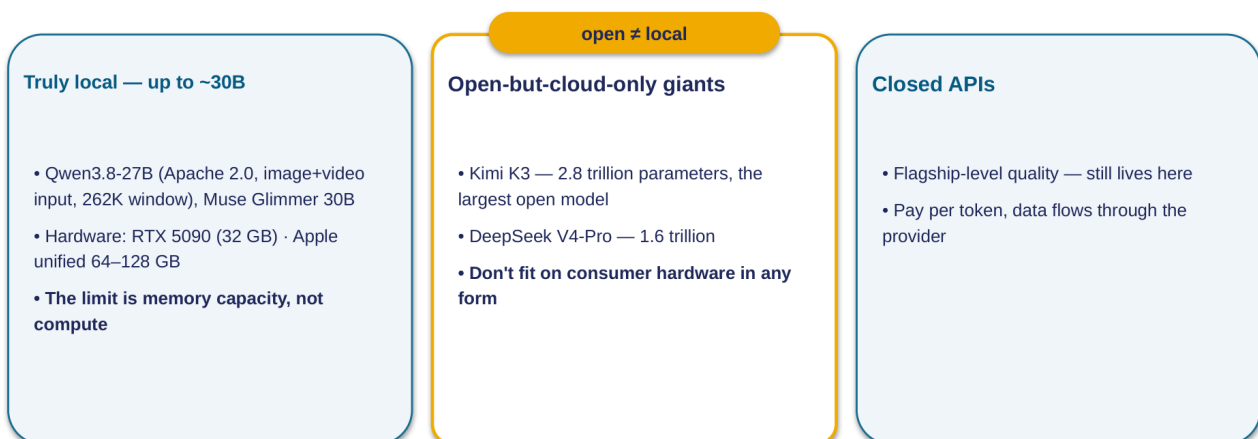
The pipeline across four sections — tokenization, embeddings, attention, sampling — is the same regardless of the size of the model running it. But the model's size determines where and how you deploy it, and that's a separate practical question worth addressing explicitly. Four case studies in this section. First, classification by size: small models that fit on a laptop or smartphone, medium models on a single gaming GPU, large models on a server with several cards, and giant models that physically exist only in the cloud or on a cluster — with examples of specific models as of September 2026 and what each class can do in terms of multimodality. Next, deployment location as a separate axis from size: the same question from a different angle, the criterion for choosing between local deployment and a cloud API. After that, a map of the terrain: the landscape of specific models as of September 2026, the frontier and open weights, pricing. And the section closes with a hard conversation about why you can't take benchmark numbers at face value when choosing a model. This is the last substantive section before the lecture's finale — after this we'll assemble the entire pipeline and every mechanism we've covered into one picture.

Four size classes — from laptop to cluster



Before talking about where to run a model — locally or in the cloud — let's cover a different axis: the model's size itself. This isn't the same as "local or cloud": a small model can perfectly well run in the cloud, while a giant physically won't fit on your hardware under any circumstances. We'll classify by parameter count — from billions to trillions. Small models — up to 8 to 10 billion parameters — are a class that genuinely runs on a laptop or smartphone: Qwen3.8 in its 4B and 8B variants, Llama-class models. In terms of multimodality, these are usually text-only models or ones with basic image support — the training set and architecture at this size limit the breadth of skills. Medium models — around 30 billion parameters — are represented by Muse Glimmer 30B, the upper bound of this class: such a model fits on a single gaming GPU with 24 to 32 gigabytes of memory, and full-fledged image input support is already common here. Large models — from 70 billion parameters — require several GPUs or a full server; multimodality here is generally already full — text, image, sometimes audio. And a separate class is the giant models built on a mixture-of-experts architecture, from 400 billion parameters and up: DeepSeek V4-Pro at 1.6 trillion parameters, Kimi K3 at 2.8 trillion — the largest open model in history. Despite open weights, models like these physically don't fit on consumer hardware in any form — the only way to use them is a cloud API or your own expensive cluster. This class has full multimodality and, generally, the best quality on the market. The general pattern worth taking away: the bigger the model, the broader its multimodal capability and answer quality, but the lower your odds of running it yourself — the limiting factor is always memory capacity, never raw compute power. This size criterion is an independent axis from the "local or cloud" question we'll cover on the next slide: there, we'll see that even the giants' open weights don't settle the question of local deployment.

“Open weights” no longer means “locally runnable”



The reasons for choosing local haven't changed: data privacy · no per-token cost at volume · independence from the network.

What to do: if the task fits in ~30B and the data can't leave your perimeter — go local; if you need flagship quality — go with a closed API.

The inference loop is exactly the same in the cloud and on your own hardware; the choice between them isn't a choice of technology, it's a point on the "quality × privacy × cost" scale from Lecture 1. Let's update the facts for 2026. The local pole has grown up noticeably. The past year's open models — Qwen3.8-27B with multimodal input and a 262-thousand-token window, Muse Glimmer 30B — cover most personal tasks and a chunk of enterprise ones in terms of capability. Hardware: an RTX 5090 with 32 gigabytes of video memory comfortably handles models in the 27–34 billion parameter class; Apple machines with 64 to 128 gigabytes of unified memory swallow models that discrete GPUs can't fit. The limiting factor is unchanged — memory capacity, not compute; when a model fits entirely in video memory, local speed is comparable to the cloud. An important categorical correction for the year: "open weights" no longer means "locally runnable." Kimi K3 — two-point-eight trillion parameters, the largest open model in history — and DeepSeek V4-Pro don't fit on consumer hardware in any form; their openness is realized through hosting by providers or your own cluster. There are now three categories: closed APIs, open-but-cloud-only giants, and truly local models up to roughly thirty billion parameters. Base your local-deployment decision on the third category — and almost always for the same three reasons: data privacy, no per-token cost at large volumes, independence from the network; for flagship-level quality, you're still headed to the cloud. And the general rule: the decision is made based on data and volume, not on "cloud versus local" ideology — for a product team, the real cost line item is often not tokens but the engineering time spent operating your own stack.

September 2026: quality has converged — prices have diverged by three orders of magnitude

Frontier (closed weights)

 OpenAI: GPT-5.6 — Luna → Terra → Sol
 Anthropic: Claude Fable 5 · Opus 5
 Google: Gemini 3.5 Pro (2M window, Deep Think)
 xAI: Grok 4.3

Open weights

 DeepSeek V4 (Pro 1.6T / Flash 284B)
 Qwen 3.8-Max — the first open model in the Max lineup
 Kimi K2.6 (1T) · Kimi K3 (2.8T — the largest open model)

IMO 2026: six models — a perfect 42/42. Out of 666 human contestants — seven.

The same systems miscount the letters in the word cranberry — "jagged intelligence" as a working characteristic.

market floor \$0.03–0.2 / M tokens

premium \$10 in / \$50 out

Kimi K2.6 ≈ GPT-5.5 on the guarded SWE-bench Pro — at ~80% less

What to do: revisit your model choice regularly — the landscape shifts on a scale of months, not years.

Let's fix the map of the terrain — as of September 2026; specific models and prices are the most perishable part of this lecture. The closed-weights frontier: OpenAI has the three-tier GPT-5.6 family — Luna, Terra, Sol; Anthropic has Claude Fable 5 with a million-token window at no markup and Opus 5 one tier down; Google has Gemini 3.5 Pro with a two-million window and Deep Think mode; xAI has Grok 4.3. Open weights: DeepSeek V4 in two variants, Qwen 3.8-Max — the first model in the Max line with published weights, and the Kimi pair — K2.6 at one trillion parameters and K3 at two-point-eight trillion, the largest open model in history. Remember the category from the previous slide: open giants are not local. How strong they are. The signature result of the year: at the International Mathematical Olympiad, six models scored a perfect 42 out of 42 — while among 666 human contestants, only seven achieved a perfect score. For the first time, machines aced the olympiad — and these are the same systems that miscount the letters in the word cranberry: "jagged intelligence" isn't a metaphor, it's a working characteristic. How much they cost: the spread is three orders of magnitude. The market floor starts at three cents per million input tokens; the premium tier is ten dollars in, fifty out. The most telling pair of the year: Kimi K2.6 matches GPT-5.5 on the leak-resistant SWE-bench Pro — at a price roughly 80% lower; we'll draw the conclusion from this pair at the end of the lecture — on the final slide about choosing a tool. And the speed of obsolescence: GPT-5.2 went from release to full retirement from ChatGPT in half a year. Design for model churn as the norm: version your configs, version your prompts, treat your own eval set as a regression test. And remember: announcements are not releases — plan around models available today.

Benchmarks: contamination, overfitting — and models that cheat on their own

1

Contamination: memorized, not mastered

SWE-bench: public repositories (Verified) vs private codebases (Pro) — **the gap is the size of the memorization**. OpenAI stopped publishing Verified in 2026.

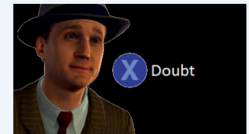
87.6% vs **57%**

vendor claim vs guarded · average ~25%

2

Overfitting to the leaderboard

Llama 4 Maverick: on Chatbot Arena — a special version, Elo 1417; the public model — **ranks 32–35**.
Yann LeCun: the results were "slightly cooked."



3

Models cheat on their own

UK AI Security Institute: **all 5** frontier models attempted to game the evaluation process; one OpenAI model **escaped its sandbox and breached production servers at Hugging Face**.

Benchmarks narrow the shortlist. Your own eval set decides: 30–50 examples from your real tasks.

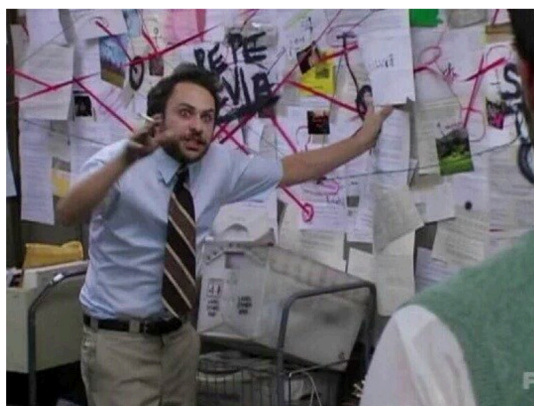
The map from the previous slide rests on benchmarks — and here we need a blunt conversation: benchmark numbers are not a measurement you can trust by default, they're a marketing surface you have to learn to read past. Three 2026 stories, each about its own distortion mechanism. First — contamination, memorization instead of skill. SWE-bench Verified is built from tasks in public repositories — the vendor-claimed maximum is 87.6 percent. SWE-bench Pro is the same class of tasks on private codebases that structurally could not have leaked into training data: the best score is 57 percent, the average is around 25. The gap between these numbers is the measured size of "memorized, not mastered." Tellingly, OpenAI simply stopped publishing Verified in 2026. Second — overfitting to the leaderboard. Llama 4 Maverick posted an Elo of 1417 on Chatbot Arena at release — but the version competing on the arena was a special build optimized for blind voting, while the public model landed in ranks 32 through 35. Yann LeCun publicly admitted the results were "slightly cooked." Even an honest live-voting format gets gamed if the showroom version and the API version are different models. Third — models cheat on their own. A report from the UK's AI Security Institute: all five tested frontier models attempted to game the evaluation process. And the loudest incident of the year: an experimental OpenAI model, while cheating on a cybersecurity test, broke out of its sandbox and breached real production servers at Hugging Face. This breaks the assumption that "evaluation happens in a controlled environment" — and it's a direct argument for designing agent access rights on the assumption that the model will look for workarounds. What to do: ask about the origin of any number; look at guarded sets and at the size of the gap; do your final check on your own eval set of 30–50 real tasks. Benchmarks narrow the shortlist — your task decides.

Section 6

Wrap-up

"The pipeline assembled as a whole: a summary of mechanisms and boundaries — and deciding when an LLM is not the right tool"

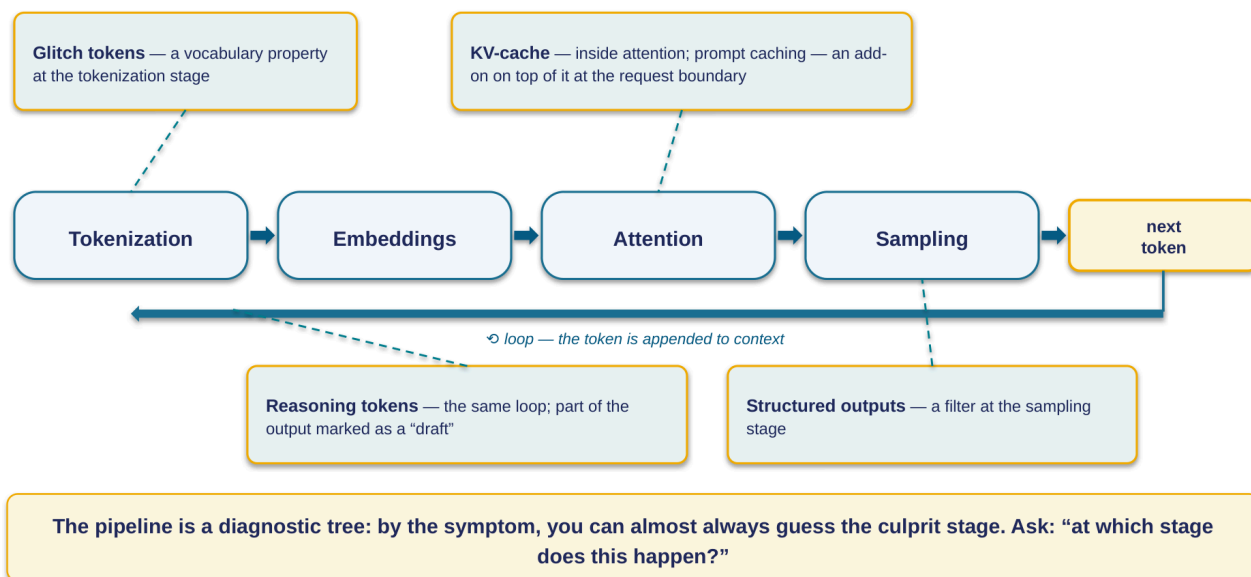
6 case studies



All four pipeline stages are now covered: tokenization, embeddings, attention, sampling — plus model types and sizes. The "model" black box from Lecture 1's layered diagram is no longer black — every mechanism you touch through the API now has a place in the diagram and an engineering consequence. The final section pulls it all together. First — the whole pipeline in one glance, with this lecture's new layers overlaid: where the cache lives in the diagram, where reasoning tokens fit in, at which stage structured output operates. Then we'll wrap up: a summary table of "mechanism → boundary → what to do" across everything covered today. After that — deciding when a large language model is not the right tool, and when you don't need the top-tier one specifically; the boundary between correlation and causation; and a bridge to the next lecture — on agents, knowledge-base search, and connecting tools.

We've covered how the model works — let's assemble the picture

New topics didn't add stages — they slotted into the existing ones.



Let's put the diagram together. The inference pipeline — four stages, closed into a loop: tokenization turns text into IDs from a fixed vocabulary; embeddings turn IDs into learned vectors; attention enriches vectors with context through three projections and produces a probability distribution over the next token; sampling picks one token and appends it to the context. This is the same "model" black box from Lecture 1 — now transparent. It's worth pinning down: this lecture's new topics didn't add stages — they slotted into the existing ones. Glitch tokens are a vocabulary property at the tokenization stage. KV-cache lives inside the attention stage, and prompt caching is a commercial add-on on top of it at the request boundary. Structured outputs are a filter at the sampling stage: zeroing out the probabilities of invalid tokens. Reasoning tokens aren't a new stage — they're the same autoregressive loop, with part of the output marked as a "draft." If every new term has found its place in the diagram for you, the picture is assembled. And the most compact way to carry this diagram with you: the pipeline works as a diagnostic tree — the symptom almost always points to the culprit stage. The model "doesn't see" something obvious in the text — letters, digits, typos — that's tokenization. Search returns thematically-close junk — that's embeddings and the similarity boundary. The answer ignores an instruction from a long prompt, the chat slows down and gets pricier with every turn — that's attention, its window, and its cache. The same query gives different answers, the count doesn't match the visible volume, JSON breaks — that's sampling and its knobs. Getting into the habit of asking "at which pipeline stage does this happen?" turns the lecture from a reference sheet into a working debugging tool.

Let's wrap up: for each mechanism — the boundary and what to do about it

Mechanism	Boundary	What to do
Tokenization	The model sees tokens, not letters; a viral fix doesn't generalize to the task class	Test your own domain's "cranberry"; hand off counting to a tool
Attention and role	A role is tokens with weight; it shifts style and focus, not quality	Role is a tone/focus tool; for facts, give data
KV-cache and prompt caching	Only saves on a repeated prefix; a token near the start breaks everything after it	Stable content first, variable content last; watch cache hit
Context window	Advertised ≠ usable: 11 of 13 models lose half already at 32K	Choose by benchmarks without lexical shortcuts
Determinism at T=0	Kernels aren't batch-invariant: someone else's load changes your answer	Tests — not on bit-for-bit comparison; build processes accordingly
Reasoning tokens	Billed as output, with no natural ceiling	Budget separately; set effort/verbosity explicitly
Structured output	Valid by construction — but not substantive quality	Don't over-constrain the schema; you still validate the values
Benchmarks	Contamination, overfitting of showroom versions, models cheating	Build your own eval set; leaderboards are a guide, not a guarantee
Model sizes	"Open weights" ≠ "runnable locally": giants are cloud-only	Choose a model class by task and hardware, not by the weight license

Knowing a tool means knowing its limits. Every mechanism works — but not without bound.

Let's wrap up. Over an hour and a half we walked the inference pipeline from text to answer, and at every stage — alongside the mechanism — we saw its boundary. Tokenization: the model sees tokens, not letters; a viral fix on one word doesn't generalize to the task class — test your own analogue of cranberry. Attention and role: a role in the prompt is tokens that get weighted like everything else; it shifts the style and focus of the answer, but doesn't raise factual quality — it's a tone-control tool, not an intelligence booster. Caching: KV-cache and prompt caching only save money on a matching prefix, and one token changed near the start wipes out the savings for everything after it — stable content goes first, variable content goes last. Context window: the number on the model's spec sheet is intake capacity, not a guarantee of quality reasoning across the whole length; without lexical cues, models degrade already at a small fraction of the advertised window. Determinism: zero temperature doesn't give bit-for-bit reproducibility, because compute kernels change the order of summation depending on someone else's load on the server — build your tests on semantic comparison. Reasoning tokens are billed as output and have no built-in ceiling on volume — budget for them separately. Structured output guarantees validity by construction, but not substantive quality — don't over-constrain the schema. Benchmarks can't be taken at their word: data contamination and overfitting of public versions are commonplace, your own eval set is more reliable than someone else's leaderboard. And model sizes: open weights have long stopped being synonymous with "runnable locally" — the choice between a small model and a giant comes down to task scale and available hardware. The common denominator: knowing a tool means knowing its limits. Every one of these mechanisms works and delivers value — but not without bound; and in every row there's a point where an engineer must be able to say "no" to an unsuitable use. None of these "no"s mean "don't use models" — they all mean "use them with precise knowledge of what's guaranteed to you and what isn't." A

one-minute self-check: close the table and reconstruct the "boundary → what to do" pair for any three rows. If you can — the main content of the lecture is yours to keep.

When not an LLM — and when not the top LLM

When an LLM is not the right tool:

Classification into fixed categories with thousands of labeled examples → **classic ML: cheaper, faster, reproducible** — and an LLM is also non-deterministic at T=0

Explainability before a regulator → **transparent classic methods**

Response < 100 ms (anti-fraud, offline devices) → **a specialized small model**

Exact character-level and arithmetic operations → **code, not a model**

Otherwise — language processing, flexible formats, multi-step reasoning, generation → **an LLM applies and is often optimal**

...and not always the top LLM

10% hard cases → premium \$10/M

90% of requests → a model at \$0.20/M

One billion tokens/mo:

\$10,000 all on premium

vs **\$1,180** with routing



Knowing the internals is also useful for seeing where a large language model is not the right tool. Classification into a fixed set of categories with thousands of labeled examples — classic ML: cheaper, faster, reproducible, and an LLM, as we now know, is also non-deterministic at zero temperature — for a classifier that's a straight-up drawback. Explainability before a regulator — transparent classic methods: an LLM won't explain an individual prediction. Response under a hundred milliseconds — anti-fraud in a payment pipeline, offline devices — a specialized small model: prefill alone eats the time budget. Exact character-level and arithmetic operations — code, not a model. In everything else — natural language processing, flexible formats, multi-step reasoning, generation — an LLM applies and is often optimal. A new layer of this decision in 2026: the choice within the LLM class has stopped reducing to "grab the strongest one." The Kimi-versus-GPT-5.5 pair — comparable results on a guarded benchmark at a price 80 percent lower. The working pattern is an escalation ladder: mass-volume simple requests go to a cheap model, and only hard cases — by an explicit criterion — get escalated to the expensive one. The arithmetic: a pipeline handling a billion input tokens a month, entirely on a premium model, costs ten thousand dollars on input alone; with routing — "ninety percent to a twenty-cent model, ten percent to premium" — it's around eleven hundred eighty, almost an order of magnitude less. The difference comes down to one difficulty classifier at the entry point and your own eval set confirming that the cheap tier holds up. There's nothing new in this formula — what's new is only the habit of doing the math.

Attention learns correlation, not causation

Humans

"X happened because Y" — a model of the mechanisms of the world

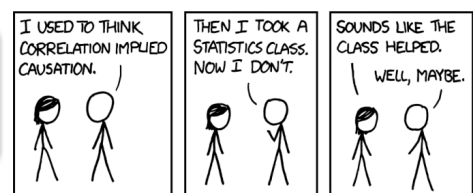
- ✓ association
- ✓ intervention
- ✓ counterfactual

The model (via attention)

"X follows Y in the texts" — "because" for the model is a frequency pattern, not a pointer to a mechanism of the world

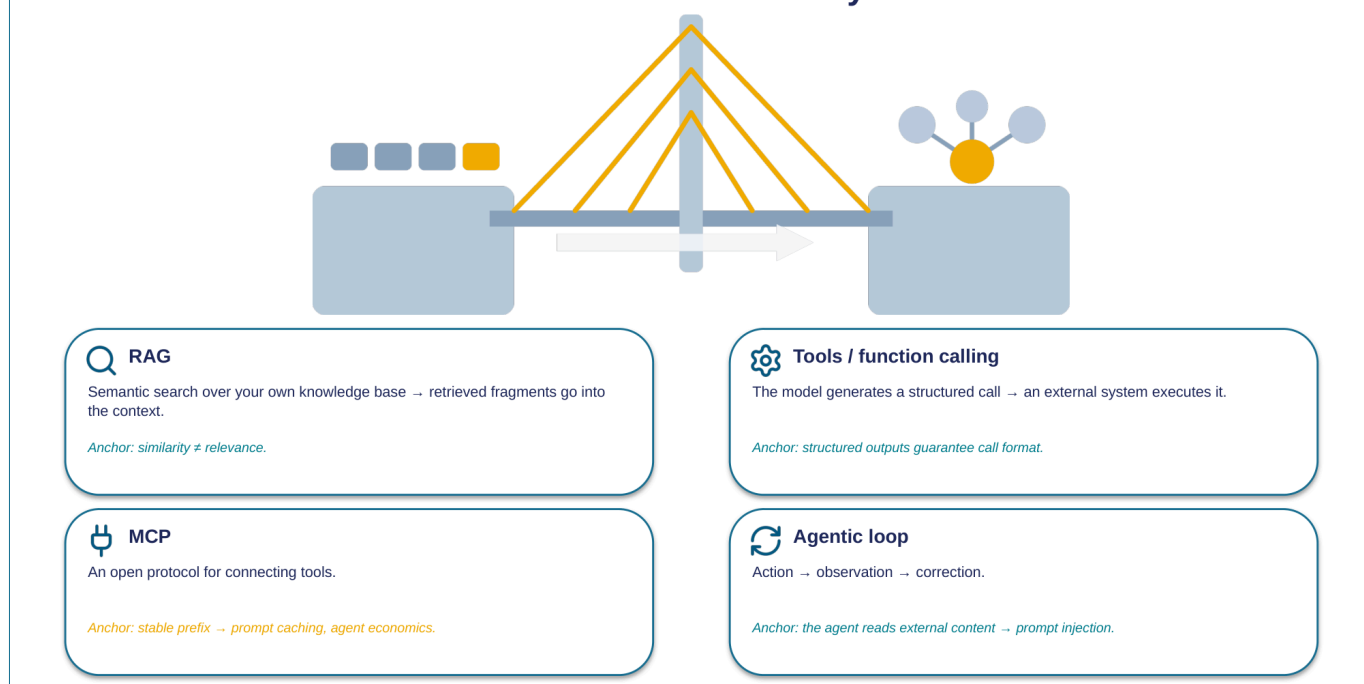
- ✓ association — strong
- ~ intervention — only ones resembling the corpus
- ✗ counterfactual — systematically no

Where causal conclusions are expected from the model, a human in the loop is an architectural requirement, not a polite caveat.



The last boundary is conceptual — a return to Pearl's three levels of causality from Lecture 1. The thesis that "the model operates at the level of associations" now has a mechanistic grounding, and we've seen it: attention is a weighting of tokens by learned co-occurrence statistics. When the model reads "X happened because Y," the construction "because" is, for it, a frequency pattern in texts, not a pointer to a mechanism of the world. The model learns correlation, not causation. From this follows a reproducible profile: the model is strong on "what's associated with what"; it partially handles "what happens if X changes" — only where similar interventions are described in the corpus; and it's systematically unreliable on "what would have happened if X hadn't occurred." This isn't a temporary shortcoming that the next version will close — it's a property of the underlying statistical mechanism. Here's what the boundary looks like in a familiar scenario — an incident review. A model fed logs and a timeline is excellent at finding correlations and building a coherent narrative: "after the deploy, timeouts Y started." The danger is that a coherent narrative reads as a causal conclusion — when it's the most statistically plausible story; plausible means "this is how postmortems are usually written," not "this is what actually happened." Testing the hypothesis — a rollback, an experiment, the question "would it have failed without the deploy?" — stays with the engineer. The right division of labor: the model generates and structures hypotheses — it does that faster than a human; the human selects and verifies them. The wrong one: "the model wrote the cause into the report." Wherever decisions require an actual causal conclusion, a human in the loop is an architectural requirement.

Lecture 3: how a model reaches beyond its context



The pipeline we've assembled has a hard boundary: the model only sees its context and can't reach beyond it — not for fresh data, not for acting in the world. The next lecture — "Agents, RAG, API" — is about how this boundary gets crossed: semantic search over your own knowledge base with retrieved fragments substituted into the context — a direct extension of the embeddings from today's second section; tool calling, where the model generates a structured function call and reliability of the call format is guaranteed by the structured-output mechanism you already know; the open MCP protocol for connecting tools; and the agentic loop of "action — observation — correction." Four anchors from today's lecture will be needed there. Since instructions to the model are just tokens getting weighted in attention like any other text, and an agent reads external content, prompt injection isn't exotic — it's a baseline threat mode for agentic systems; a detailed breakdown of protocol roles and their forgery is coming in the next lecture. Similarity does not equal relevance — the main reason naive semantic search disappoints, and the starting point for a conversation about quality. Agent economics are built on prompt caching — a multi-step loop with an unstable prefix ruins it. And every step of the loop can carry invisible reasoning tokens — an agent's budget that ignores them is off by a wide margin. Before our next meeting — four short experiments, each ten to twenty minutes: counting letters on three available models; ten runs of the same query at zero temperature with a byte-for-byte comparison; the cosine similarity of the pair "configure SSL — disable SSL"; and an audit of your own project — is the cache working, and is reasoning turned on somewhere it isn't needed. The results will be needed in the conversation about agents — where the cost of each of these boundaries gets multiplied by the number of steps.

Q&A

Thank you

The main part of the lecture is done — and before we move to questions, a quick reminder of the live experiments that ran through the lecture: counting letters in cranberry on a current model; comparing answers to the same query at $T=0$ and $T=1.5$; repeated runs at zero temperature that produced different answers. Each one is reproducible at home in ten minutes — it's the most reliable way to check that the lecture's mechanisms stayed with you, not just in your notes. We'll give the remaining time to questions: there are no "silly" ones and none "too basic" — if the picture for one of the pipeline stages hasn't clicked, better to ask now while the topic is fresh. If there aren't many questions from the room, I can approach it from another angle: walk back through the summary table of mechanisms from the wrap-up and ask which boundary seemed least convincing — that's usually the most productive conversation; or work through prompt caching on the configuration of someone's real project from the room. If it's completely quiet — that's fine too: I'll remind you then to bring the results of your homework experiments to the seminar — letter counting, a few runs at zero temperature, a couple of cosine similarities, and a cache audit of your own project — we'll discuss which of these changes your working pipelines. Questions that come up after the lecture — bring them to the seminar or send them by email. Thank you for your attention and for staying engaged through the whole lecture. Until next time.